

Projet de semestre

Ecole supérieure

Électronique
PROJ
SLO2

Maquette Bille 2D

Réalisé par :

Mathieu Bucher

A l'attention de :

M. Bovey
M. Castoldi
M. Moreno

Dates :

Début du projet : 15 Novembre 2024
Fin du projet : 13 juin 2025

Table des matières

1. Cahier des charges	5
1.1. Cahier des charges de base :	5
1.2. Ajout au cahier des charges :	5
1.3. Choix effectués :	5
2. Schéma bloc	6
2.1. Explication des blocs.....	6
2.1.1. Bloc alimentation externe	6
2.1.2. Bloc Alimentation interne.....	7
2.1.3. Bloc microcontrôleur.....	7
2.1.4. Bloc interface de réglage.....	7
2.1.5. Bloc Touch Screen contrôleur	8
2.1.6. Ordinateur avec application.....	8
2.1.7. Servomoteurs.....	9
2.1.8. Bloc touchscreen.....	9
2.2. Estimation des coûts	9
3. Conclusion pré-études.....	9
4. Design.....	10
4.1. Modifications du schéma bloc	10
4.2. Choix des composants.....	11
4.2.1. Alimentation Externe	11
4.2.2. Alimentations 5V	12
4.2.3. Alimentation 3.3V	13
4.2.4. Microcontrôleur	13
4.2.5. Interfaces	15
4.2.6. Contrôleur du touchscreen	16
4.2.7. USB vers UART	18
4.2.8. Points de mesures pour les signaux importants	18
5. Concept software	19
6. Conclusion design	20
7. Modifications de la schématique du design	21
7.1. Alimentation 5V.....	21
7.2. Modifications du bloc USB vers UART	21
7.3. Mesures du courant dans les moteurs	22
7.4. Modification dans le bloc du contrôleur du plateau résistif.....	22
8. Hardware.....	23
8.1. PCB	23

8.1.1.	Placement	23
8.1.2.	Routage	23
8.1.3.	2D	24
8.1.4.	3D	25
8.1.5.	PCB monté.....	25
9.	Correction hardware.....	26
9.1.	Alimentation 5V.....	26
9.1.1.	Inductances.....	26
9.2.	Connection plateau résistif	28
10.	Mise en service et mesures	28
10.1.	Tension d'alimentation	28
10.1.1.	Alimentation 3.3V	28
10.1.2.	Alimentation 5V	29
10.1.3.	Alimentation 12V	30
10.2.	Mesure communication SPI entre le contrôleur et le microcontrôleur	30
10.3.	Mesures des signaux PWM.....	32
10.4.	Mesure de la communication UART	33
10.5.	Courant.....	34
10.5.1.	Consommation totale	34
11.	Firmware.....	35
11.1.	Fonctions	35
11.1.1.	Lecture plateau	35
11.1.2.	Réglage P	35
11.1.3.	Réglage I.....	36
11.1.4.	Réglage D	36
11.1.5.	Règlage PID.....	36
11.1.6.	Génération PWM.....	37
11.1.7.	Envoie UART	37
11.1.8.	Réception UART	37
11.1.9.	Gestion des boutons	37
11.1.10.	Affichage	38
12.	Software	38
13.	Amélioration.....	38
13.1.1.	Chose fonctionnelle à améliorer	38
13.1.2.	Choses manquantes à ajouter.....	38
14.	Aspect environnemental du projet.....	38
15.	Aspect social.....	39

16.	Aspect économique	39
17.	Conclusion	40
18.	Annexes.....	41
18.1.	Datasheets.....	41
18.2.	Schéma.....	44
18.3.	Listing firmware.....	44
18.3.1.	Listing app.c	Erreur ! Signet non défini.
18.3.2.	Listing app.h.....	Erreur ! Signet non défini.
18.3.3.	Listing main.c	Erreur ! Signet non défini.
18.3.4.	Listing Mc32GestTSC2046.c.....	Erreur ! Signet non défini.
18.3.5.	Listing Mc32GestTSC2046.h.....	Erreur ! Signet non défini.
18.4.	Prix PCB Eurocircuits.....	44

1. Cahier des charges

1.1. Cahier des charges de base :

- Récupérer la partie mécanique de la maquette et ajouter une électronique permettant le contrôle des servomoteurs.
- Ajouter une interface physique pour le réglage des facteurs de correction proportionnel, intégrateur, dérivateur.
- Créer une application en C# pour le même principe que l'interface physique communiquant via USB ou Ethernet sur un ordinateur.
- Choisir le type d'alimentation : 5V (USB C), bloc USB (µUSB) ou bloc d'alimentation 9V ou 12V.

1.2. Ajout au cahier des charges :

- Sélectionner le microcontrôleur pour le projet (PIC32 ou STM32).
- Obtenir la position de la bille sur le touch sensor avec ou sans le contrôleur de base.
- Choisir la commande des moteurs.
- Choisir la meilleure façon d'interfacer le réglage.
- Choix de faire 1 ou plusieurs PCB.

1.3. Choix effectués :

- Le microcontrôleur utilisé sera le PIC32 car c'est celui qui est utilisé lors des TP en deuxième année.
- L'interface physique se fera via 4 boutons et un écran LCD. Plus simple pour visualiser et manipuler les valeurs.
- La détection de l'emplacement de la bille se fera grâce à un TSC2046. Composant venant de texas instrument très peu de composant supplémentaire pour l'utiliser et permet de communiquer en SPI vers le microcontrôleur.
- Faire 2 PCB (un avec alimentation et commande moteur + microcontrôleur et un pour la commande du touchscreen).
- Pour la communication de l'application en C# vers le PCB, utilisation de l'USB car cela a été suggéré par monsieur Bovey.
- Pour l'alimentation, utilisation du même montage que le projet de base avec deux régulateurs DC/DC.

2. Schéma bloc

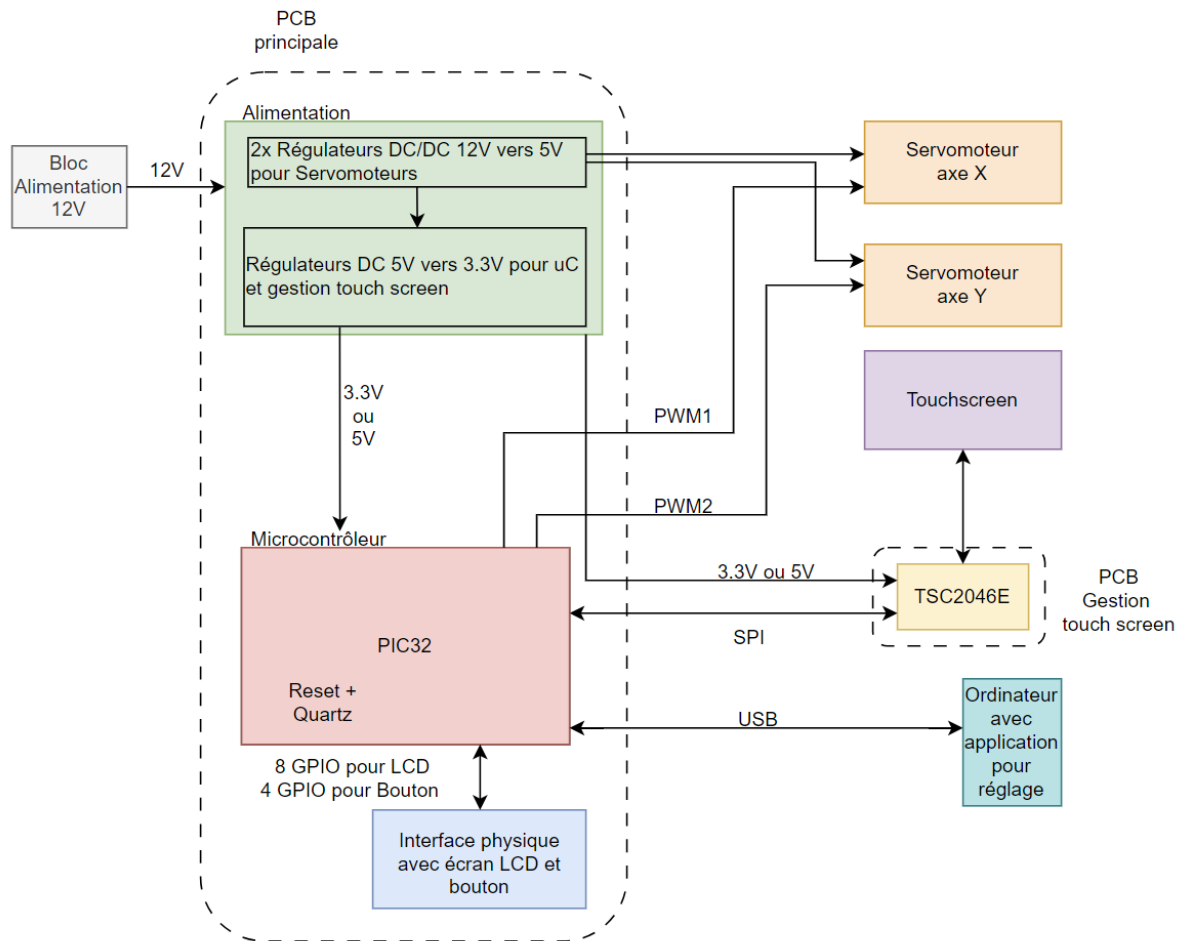


Figure 1 Schéma blocs

2.1. Explication des blocs

2.1.1. Bloc alimentation externe

Ce premier bloc représente l'alimentation du projet.

Pour alimenter le projet j'avais plusieurs choix à ma disposition :

- Alimenté via l'USB d'un ordinateur par exemple.
- Bloc d'alimentation secteur vers un micro USB.
- Bloc d'alimentation secteur 9V.
- Bloc d'alimentation secteur 12V.

2.1.1.1. Alimenté via l'USB d'un ordinateur

Cette option n'est pas la meilleure car l'alimentation d'un USB peut fournir un courant maximum de 500 mA pour un USB2.0 et 900 mA pour un USB3.0.

2.1.1.2. Bloc d'alimentation secteur vers un micro USB

Le bloc d'alimentation secteur est la seconde option proposée. Il serait possible d'en utiliser un mais les blocs secteurs 5V ne fournissent pas assez de courant.

2.1.1.3. Bloc d'alimentation 9 et 12V

C'est le choix sur lequel je vais me porter. Maintenant il reste à savoir 9 ou 12V. Les fichiers du projet de base d'ACROME utilisait une alimentation 12V 5A. Je vais donc récupérer l'ancienne alimentation secteur pour la nouvelle version.

2.1.2. Bloc Alimentation interne

Dans ce bloc, nous retrouvons les régulateurs que je vais utiliser pour alimenter les 2 servomoteurs, le microcontrôleur, le contrôleur qui permet de communiquer l'emplacement de la bille sur le touch screen, l'USB et l'interface de réglage.

2.1.2.1. LM2596S

Le LM2596S est régulateur linéaire permettant de réguler ma tension 12V du bloc secteur à 5V pour alimenter mes servomoteurs. Je l'ai choisi car il était présent dans l'ancien projet.

2.1.2.2. MAX1793

Après avoir réguler mon 12V en 5V, j'utilise un régulateur pour obtenir du 3.3V afin d'alimenter mon microcontrôleur PIC32. Je vais aussi pouvoir me servir de cette alimentation pour alimenter l'IC qui retranscrit la position de la balle.

2.1.3. Bloc microcontrôleur

Dans ce bloc nous allons retrouver : le microcontrôleur, les composants servant au bon fonctionnement de celui-ci, Quartz et Reset.

2.1.3.1. PIC32MX795F12L

J'ai décidé d'utiliser ce microcontrôleur car c'est celui que nous utilisons lors des travaux pratiques de MINF. Ce microcontrôleur à bien assez d'I/O pour accueillir les 4 boutons et le LCD de l'interface de réglage. Il est aussi capable d'accueillir un périphérique USB et de fournir deux canaux pour les PWM pour le contrôle des servomoteurs.

2.1.3.2. Quartz

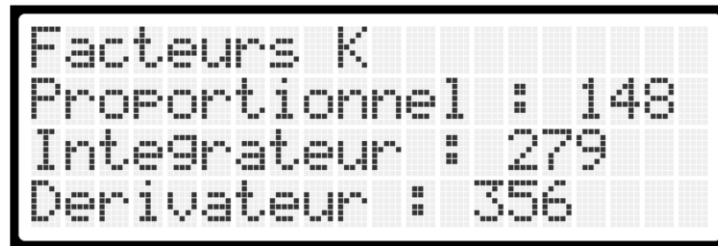
Je vais ajouter un oscillateur externe pour le microcontrôleur afin de garantir une fréquence de fonctionnement stable est sur laquelle je pourrais me baser pour mes timers.

2.1.4. Bloc interface de réglage

Il s'agit de l'interface physique du projet qui permettra de régler les facteurs proportionnel, intégrateur et dérivateur. Cette interface se fera via un écran LCD et quatre boutons pour modifier les valeurs.

2.1.4.1. Le LCD

Le LCD servira d'affichage pour les valeurs des facteurs de réglages voici un exemple :



Je vais utiliser un LCD avec 4 lignes et avec 20 caractères par colonne. Les 3 valeurs de facteurs K seront modifiable via quatre boutons poussoir.

2.1.4.2. Les boutons

Il y aura quatre boutons directionnels : haut, bas, gauche et droite. Ces boutons serviront à effectuer les corrections lors des tests de réglages.

2.1.5. Bloc Touch Screen contrôleur

Dans ce bloc/PCB, nous allons retrouver le chip qui permet de récupérer les informations sur le touch screen. Pour cette partie, j'avais plusieurs choix qui s'offraient à moi :

- Réutilisation du PCB développé par ACROME
- Intégré un contrôleur équivalent sur le PCB principale
- Développer un second PCB comme celui de l'ancien projet

2.1.5.1. Réutilisation du PCB développé par ACROME

Sur le projet développé par ACROME, il y a déjà un PCB avec un contrôleur fixé sous le touchscreen et il communique en USB vers le reste du projet. Le problème c'est que ce PCB a son propre software avec un contrôleur pour le touchscreen et une interface USB. Je n'ai donc pas accès au firmware du contrôleur et je vais devoir rajouter une partie pour décoder l'USB provenant du PCB. Cela n'est pas arrangeant car cela me rajoute un décodage

2.1.5.2. Intégré un contrôleur équivalent sur le PCB principale

La seconde possibilité serait d'ajouter un contrôleur équivalent sur le PCB principale. Cela permettrait d'avoir un seul PCB pour le projet et donc réduire peut-être un peu les coûts de productions. Malheureusement la natte du touch screen est un peu trop courte et contraindra le placement du PCB principal.

2.1.5.3. Développer un second PCB comme celui de l'ancien projet

La dernière option consiste à créer un second PCB qui accueillera le module et qui sera placé sous le plateau via les trous de fixation de l'ancien PCB. Cela me permet de gagner un peu de place sur mon PCB principale. Je vais donc choisir cette option.

2.1.6. Ordinateur avec application

Ici nous retrouvons l'interfaçage des correcteurs proportionnel, intégrateur et dérivateur via un software programmer en C#. L'application communiquera via un USB vers le microcontrôleur.

2.1.7. Servomoteurs

Il y a deux servomoteurs qui permettent de gérer les axes X et Y du plateau. Ces moteurs ont un axe de rotations de 180°.

2.1.8. Bloc touchscreen

Ce bloc contient un touchscreen résistif de 1200x800 pixels posé sur un plateau de 375x310mm.

2.2. Estimation des coûts

Pour me donner un ordre d'idée, j'ai estimé ce que pourrait coûter la réalisation de ce projet. J'ai estimé la taille que feront les deux PCB et j'ai calculé le coût avec l'outil à disposition d'euro-circuits. J'ai aussi ajouté le prix des composants que je pense utiliser. Je me retrouve donc avec ceci :

Composants	Quantité	Prix par pièce [CHF]	Total par pièce [CHF]	Total [CHF]
LM2596S	2	5.07	10.14	130.84
MAX1793	1	5.27	5.27	
TSC2046	1	1.43	1.43	
PCB Principale	1	65	65	
PCB TouchScreen	1	49	49	

3. Conclusion pré-études

Cette phase de pré-étude m'a permis d'éclaircir les différents points du cahier des charges. J'ai notamment pu étudier les différentes problématiques qui s'ajouteront en réalisant mon projet. J'ai aussi pu effectuer des recherches sur les différents composants que je peux utiliser et sur les caractéristiques nécessaires pour les sélectionner.

Maintenant, je dois faire le choix des composants que je vais utiliser et commencer le design et la schématique.

Lausanne, le 6 décembre 2024

Bucher Mathieu

4. Design

4.1. Modifications du schéma bloc

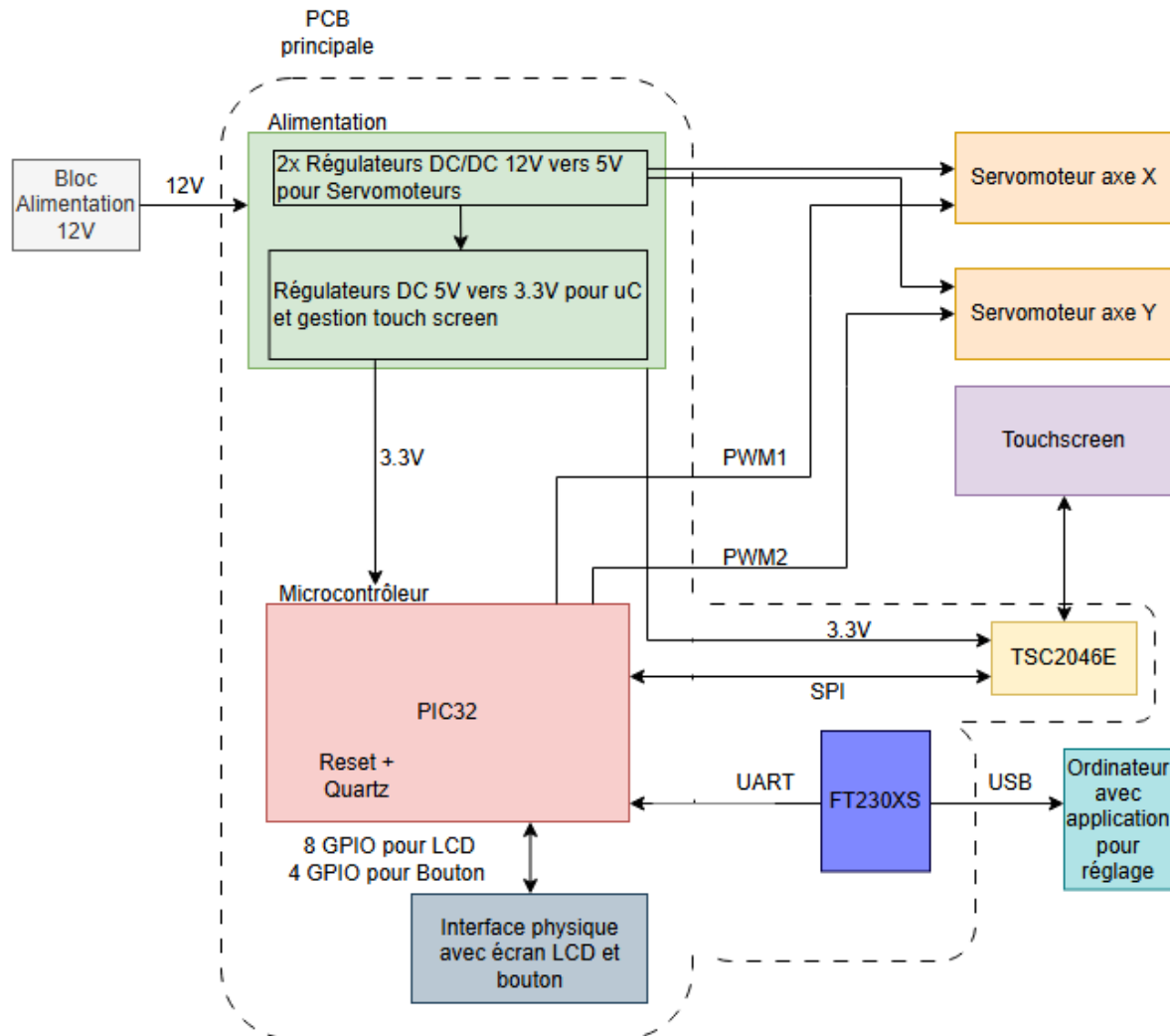


Figure 2 Nouveau schéma bloc

Il y a eu quelques modifications, notamment l'ajout d'un module FT230X. J'ai aussi placé le contrôleur pour le touchscreen sur le PCB principale. Les modifications seront développées dans les points suivants.

4.2. Choix des composants

4.2.1. Alimentation Externe

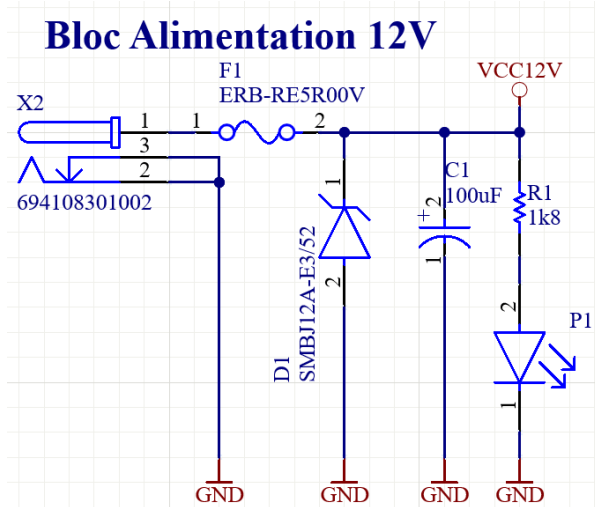


Figure 3 Schéma partie Alimentation 12V

Pour l'alimentation de la carte, j'avais choisi de récupérer le bloc d'alimentation 12V 5A Estone ES-9487 de l'ancienne version du projet. J'ajoute un connecteur jack 5.5mm Würth N° 694108301002 qui me permet de brancher le bloc d'alimentation. Ce connecteur est fiable et sera compatible avec d'autres bloc secteurs. Pour protéger l'ensemble de mon circuit, j'ai ajouté un fusible F1 32V 5A contre les surintensités et les court-circuits.

J'ai ajouté une diode TVS D1 (SMBJ12A) avec une tension nominale de 12V pour protéger le circuit contre les pics de tensions.

J'ai ajouté un condensateur C1 de 100uF pour lisser et stabiliser la tension de sortie du bloc.

La led jaune P1 permet d'indiquer si le circuit est mis sous tensions. La résistance R1 sert à limiter le courant passant dans la led à 5mA :

$$R1 = \frac{(V_{cc} - V_F)}{I_{Led}} = \frac{12 - 2.1}{5 \cdot 10^{-3}} = 1.98k\Omega \rightarrow E24 \rightarrow 2k\Omega$$

4.2.2. Alimentations 5V

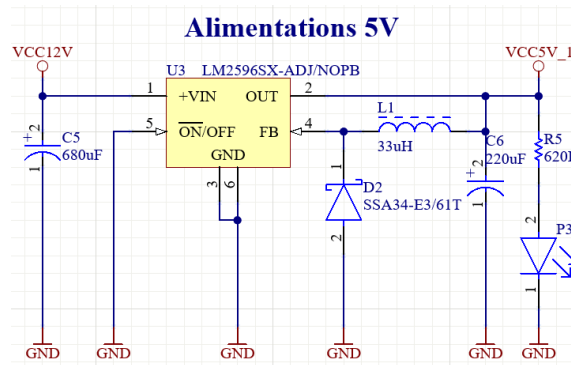


Figure 4 Schématique alimentation 5V

Concernant l'alimentation des servomoteurs, j'utilise deux convertisseurs buck 5V/3A LM2596 de chez Texas Instrument. Etant donné qu'un servomoteur consomme environs 1.57A au lancement de la rotation, j'avais besoin d'avoir une alimentation qui pourrait délivrer assez de courant. Cette tension 5V me permet aussi d'alimenter l'alimentation 3.3V.

Pour les composants autour de ce LM2596 je me suis référé au schéma recommandé par le constructeur car il répond à mes critères :

Typical Application (Fixed Output Voltage Versions)

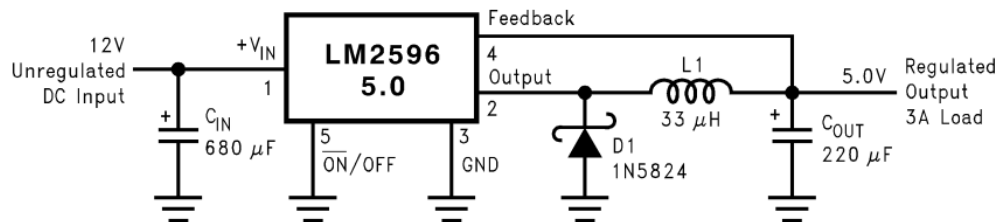


Figure 5 Schéma recommandé dans la datasheet de Texas Instruments

J'ai dû changer de diode schottky car celle recommandée était obsolète. J'ai orienté mon choix vers une diode schottky SSA34 qui a des caractéristiques similaires supportant 40V en tension inverse et 3A en courant. Comme pour le 12V, une led permet d'indiquer si le régulateur fonctionne bien.

$$R5 = \frac{(V_{cc} - V_F)}{I_{Led}} = \frac{5 - 2.1}{5 \cdot 10^{-3}} = 580\Omega \rightarrow E24 \rightarrow 620\Omega$$

Pour le microcontrôleur, mon choix c'est porté sur un PIC32MX230F256D. J'avais besoin de 22 ports au minimum ce qui me limitait vraiment en utilisant un PIC32 avec 28 pins. Les 44 pins m'offrent une meilleure flexibilité pour mapper les signaux sur mes pins. Son boîtier TQFP est compact.

Grâce à la datasheet de ce PIC32, j'ai pu mapper mes différents signaux sur les broches du microcontrôleur. Je me suis servi des pages 133 à 136 de la datasheet du PIC voir *annexes 7.1*. Nous retrouvons donc les signaux UART1_RX et UART1_CTS sur les ports correspondant au microcontrôleur. Ces 2 signaux sont pull-up pour éviter les erreurs de communication lors du démarrage du PIC32. Les signaux UART1_TX et UART1_RTS sont aussi connectés sur les pins alloués à l'UART.

Ensuite pour le SPI, les signaux sont aussi mappés sur les bonnes pins alloués au SPI1 du PIC32.

Les signaux PWM pour les 2 moteurs sont mappés sur les OC1 et OC2. Les signaux pour la programmation du PIC32 sont mappés sur les pins PGED1 et PGEC1.

J'ai utilisé des pins du PORTB pour l'utilisations des boutons poussoirs de l'interfaces et le PORTA et le PORTC pour les signaux du LCD.

Reset µC

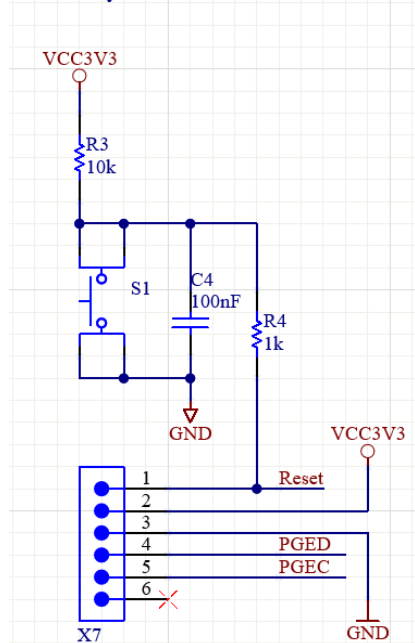


Figure 8 Partie reset et connecteur de programmation

Les signaux pour la programmation du PIC32 sont mappés sur les pins PGED1 et PGEC1 et sur le connecteur de programmation X7. Le connecteur X7 est une barette de picots pour faciliter la connexion du câble du debugger snap. Il y aussi un bouton poussoir qui permet de reset le PIC32 via la pin 18.

J'ai ajouté des condensateurs de découplage de 100nF sur chaque pin d'alimentation du PIC32. Pour la pin VCAP, j'ai ajouté comme recommandé dans le datasheet un condensateur de 10 µF pour le découplage.

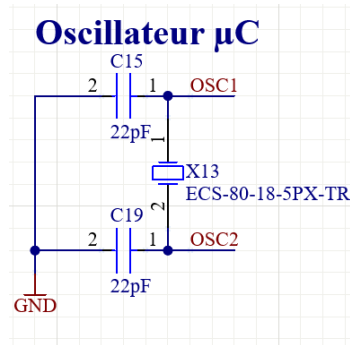


Figure 9 Partie oscillateur du PIC32

J'ai aussi ajouté un quartz de 8 MHz, avec deux condensateurs de 22pF pour améliorer la précision de mes signaux PWM afin de garantir un réajustement précis de la plateforme de balancement.

4.2.5. Interfaces

L'interface est composée d'un écran LCD qui permettra d'afficher les valeurs des facteurs de corrections du PID et de boutons poussoir qui permettront de modifier les valeurs de ces facteurs.

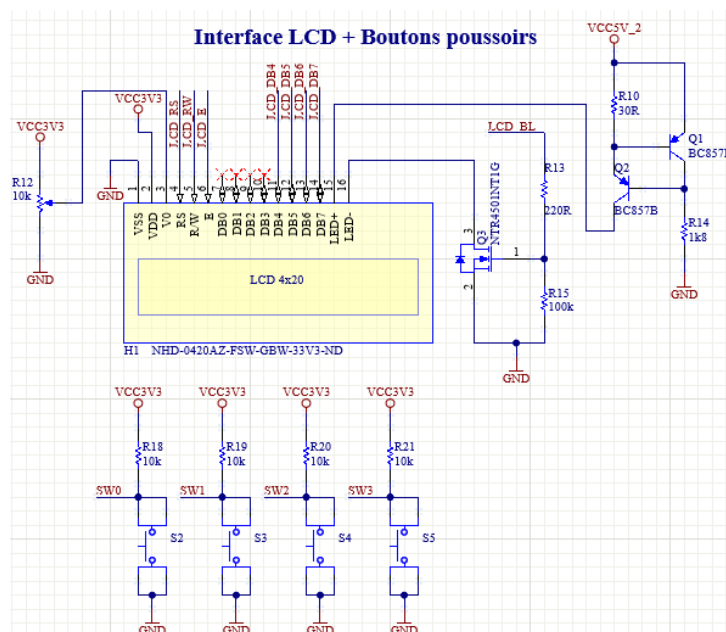


Figure 10 Partie interfaces

L'écran LCD NHD-0420AZ-FSW-GBW-33V3 fournit une grande surface d'affichage ce qui me permet d'indiquer clairement les 3 facteurs et leurs valeurs. De plus, il s'alimente en 3.3V. Nous avons un potentiomètre R12 de 10kΩ qui permet de régler le contraste de l'écran. Les quatre boutons poussoirs SW0 à SW3 sont pull-up pour maintenir un niveau logique haut lorsqu'ils ne sont pas pressés. Cette schématique est récupéré du projet 11020E (Kit de programmation PIC SK32MX).

Le montage avec les transistors Q1 et Q2 permet de limiter le courant pour le rétroéclairage. La résistance R15 de 100kΩ empêche le transistor Q3 de commuter sans un signal d'activation de LCD_BL.

4.2.6. Contrôleur du touchscreen

Il y a eu une légère modification pour cette partie du schéma. Il était prévu qu'un PCB dédié soit créer afin de le placer sous le plateau à la place de l'ancien PCB.

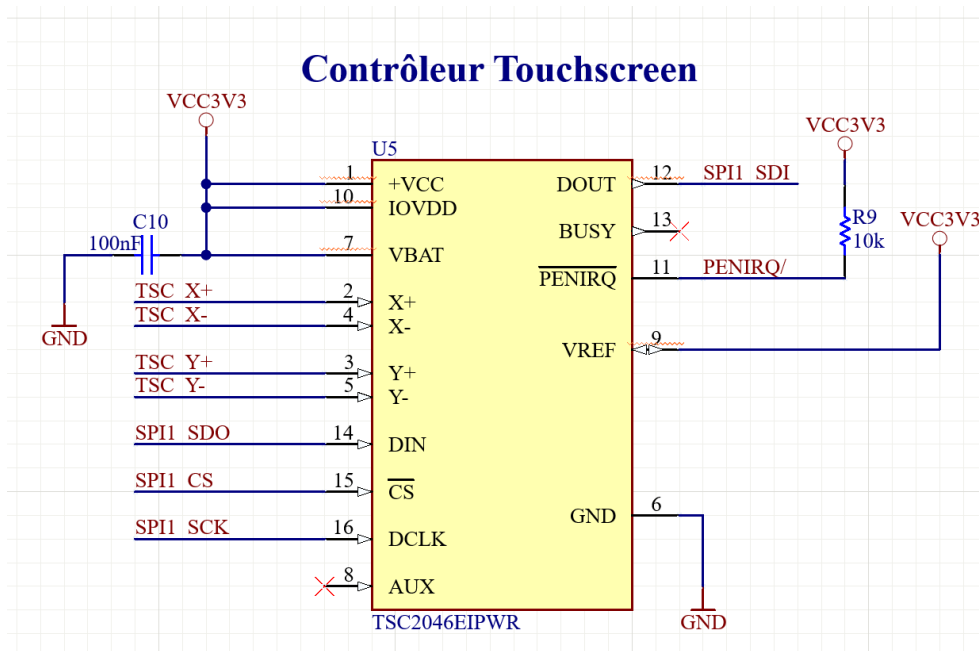


Figure 11 Schéma partie touchscreen

J'utilise le TSC2046E de chez Texas Instruments pour contrôler le touchscreen qui me renverra la position de la balle sur le plateau. Il permet de récupérer les signaux X+, X-, Y+ et Y- du touchscreen et de renvoyer ces informations en SPI. J'ai donc connecté les signaux SPI vers le PIC32MX. Il peut être alimenté 3.3V comme la majorité de mon circuit. De plus la pin PENIRQ/ permet d'envoyer une interruption s'il y a une pression sur l'écran. Je l'utiliserai pour indiquer si la balle se trouve sur le plateau ou non. Le constructeur recommande d'ajouter un condensateur de 100nF sur VCC+, IOVDD et VBAT.

J'ai ajouté une résistance de pull-up de 10kΩ sur PENIRQ/ car elle était active basse, cela permettra d'avoir un niveau logique haut lorsqu'elle n'est pas activée.

Les signaux TSC_X+, TSC_X-, TSC_Y+, TSC_Y- sont connectés directement au touchscreen via un câble connecter entre les picots sur la carte et l'écran.

La pin VREF sert de référence pour l'ADC interne du TSC2046E. Je la connecte au 3.3V afin de garantir des mesures précises sur les signaux du TSC.

Lors de la réalisation de cette partie du schéma, j'ai pu lire ceci :

In the specific case of use with a resistive touch screen, care should be taken with the connection between the converter and the touch screen. Although resistive touch screens have fairly low resistance, the interconnection should be as short and robust as possible. Longer connections are a source of error, much like the on-resistance of the internal switches. Likewise, loose connections can be a source of error when the contact resistance changes with flexing or vibrations.

Figure 12 Extrait de la datasheet du TSC2046E

Cet extrait indique qu'il est préférable d'avoir la connexion entre le screen et le contrôleur la plus courte possible et d'éviter de connecter cela via un câble flexible ce qui pourrait induire des erreurs.

Après discussion avec M. Moreno, il m'a suggéré de rester sur le plan de base mais d'ajouter un connecteur avec les signaux SPI et l'alimentation pour pouvoir au pire des cas créer un petit PCB pour corriger les potentiels erreurs qui pourrait être induite par l'idée de base.

J'ai donc deux connecteurs pour l'option de base et un qui pourra me servir en cas de problème.

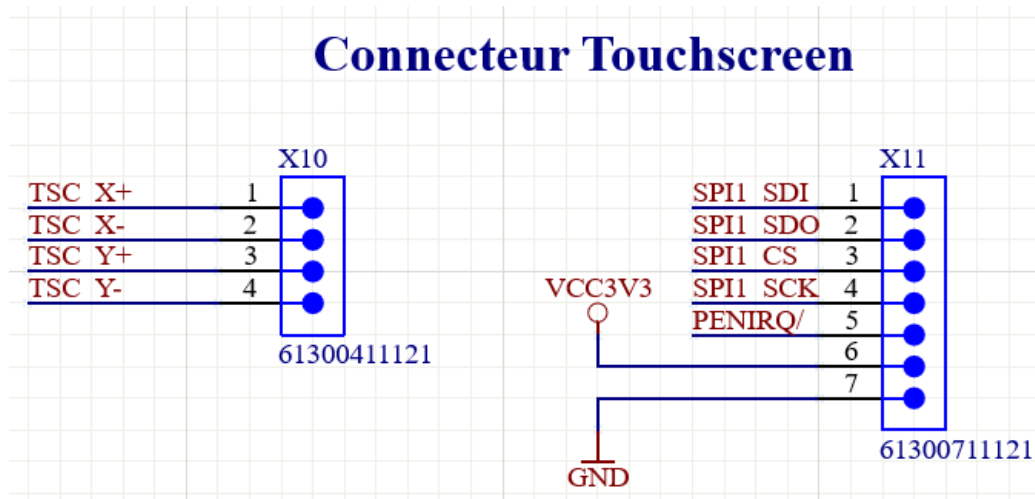


Figure 13 Connecteurs touchscreen

4.2.7. USB vers UART

Cette partie du schéma est complètement nouvelle par rapport à ma pré-étude. J'ai rajouté un module FT230XS qui va me permettre de transcrire les signaux USB vers de l'UART pour faciliter le décodage des données venant de l'ordinateur et vice versa.

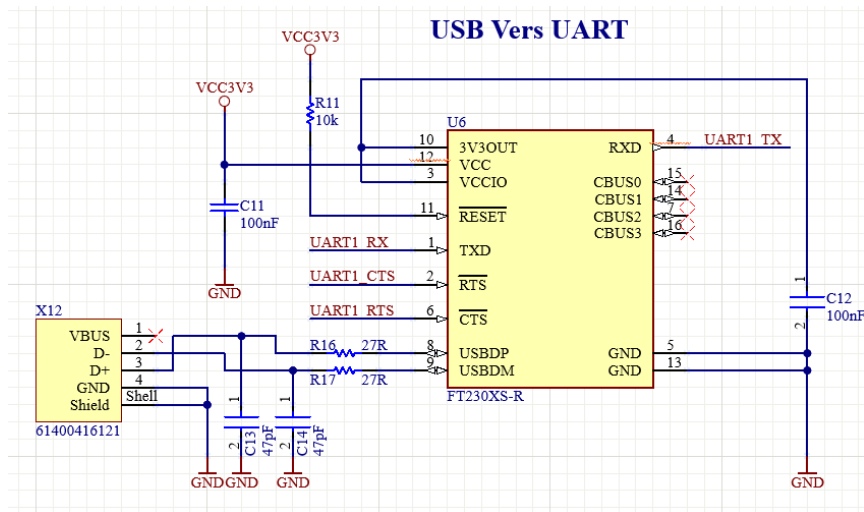


Figure 14 Schéma partie USB vers UART

J'utilise d'abord un connecteur USB de chez Würth pour connecter l'ordinateur. La ligne VBUS n'est pas connecté car l'alimentation du module vient du 3.3V de mon PCB.

Les lignes D+ et D- sont connectés en série avec des résistances de 27Ω comme la schématique du constructeur. Ces résistances servent à respecter l'impédance des lignes USB. Elles permettent d'éviter les réflexions notamment. Sur ces mêmes lignes, il y des condensateurs de 47pF pour filtrer les hautes fréquences. Ce sont des valeurs typiques recommandés par FTDI. J'ai rajouté une résistance de pull-up sur la pin reset pour garantir un niveau logique haut. J'ai aussi ajouté un condensateur de découplages de 100nF sur l'alimentation du module.

Puis nous avons les lignes de communication UART qui permettront au PIC de communiquer avec l'application sur l'ordinateur.

4.2.8. Points de mesures pour les signaux importants

Pour faciliter les tests lors de mon projet, les signaux principaux tels que l'UART ou les alimentations seront câblés sur des picots de mesures.

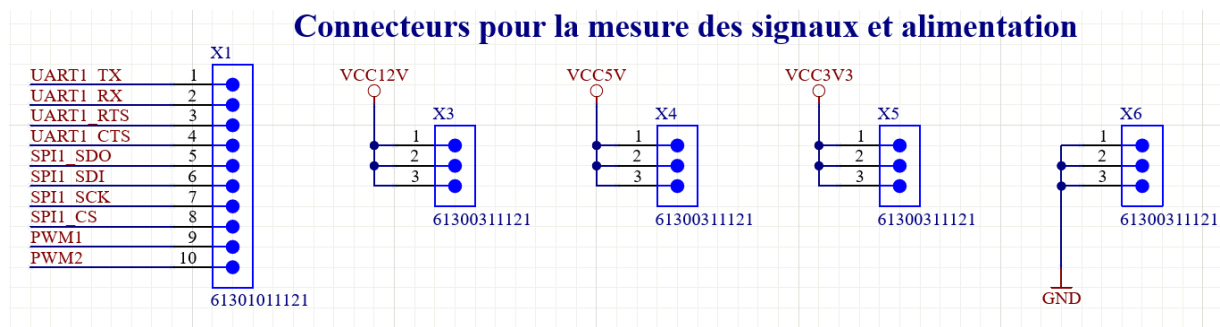


Figure 15 Point de mesure

5. Concept software

Pour le concept software, il y aura deux programme la partie firmware sur le PIC32 et la partie software via l'application en C#.

Pour la partie firmware, le microcontrôleur commencera par initialiser chaque périphérique (Timer, UART, SPI, PWM). Ensuite, un écran de démarrage se lancera. On affichera ensuite le menu de configuration des facteurs de réglages.

L'utilisateur pourra naviguer entre les différentes options via les 4 boutons poussoirs. L'application permettra à l'utilisateur de configurer ces facteurs via l'USB->UART tout en enregistrant les valeurs de la position de la balle.

Dès le moment où la balle est posée sur la plateforme, le programme continuera de lire la position tant que la balle n'est plus en contact avec le plateau. Pendant que la balle touche le touchscreen, le PIC32 va récupérer les valeurs et faire varier les signaux PWM pour faire tourner les moteurs afin de recentrer la balle. L'interface avec les boutons poussoir ou l'application permettront de régler précisément les facteurs de corrections.

L'application en C# permettra d'envoyer les différents facteurs pour la régulation PID du projet via USB. L'application pourra aussi récupérer les positions de la balle et les enregistrés dans un fichier .csv par exemple.

6. Conclusion design

Ce rapport de conception montre la réflexion derrière les choix que j'ai effectué lors de la création de ce schéma tout suivant la ligne directrice de la pré-étude. Néanmoins certains points de cette pré-études ont été modifiés ou même des nouveautés se sont ajoutées. Notamment l'ajout du module FT230XS qui va faciliter le décodage des informations provenant du PC vers le microcontrôleur.

L'alimentation de mon projet comporte trois tensions différentes 3.3V, 5V et 12V. Chaque partie garantit une alimentation stable et une protection contre les surintensités. Grâce au régulateur buck, je peux assurer un courant suffisant pour les servos moteurs tout en alimentant d'autres parties du schéma. L'ajout du fusible et des diodes TVS ont renforcés la sécurité.

L'utilisation du PIC32MX230F256D m'as permis d'avoir un peu plus de flexibilité sur le mapping des périphériques. De plus, il est capable de gérer plusieurs périphériques en même temps. L'écran LCD et les boutons poussoirs offrent une interface plus intuitive.

L'ajout des picots pour faire des mesures de signaux me permettront de déterminer un potentielle problème de manière efficaces.

La conception software me permet d'avoir une idée de base pour attaquer la programmation du PIC32. De plus, elle me sera utile pour développer l'application utilisateur en C#.

Lausanne, le 27 janvier 2025

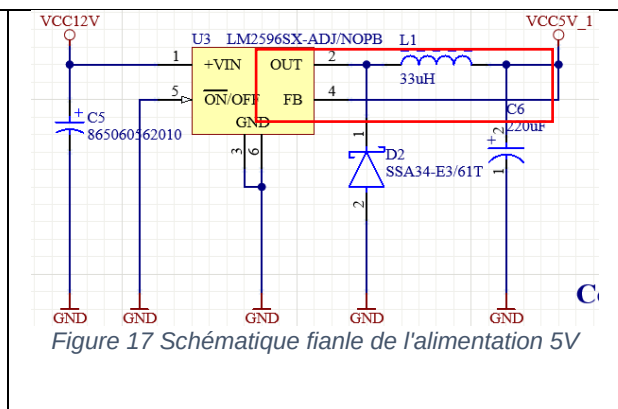
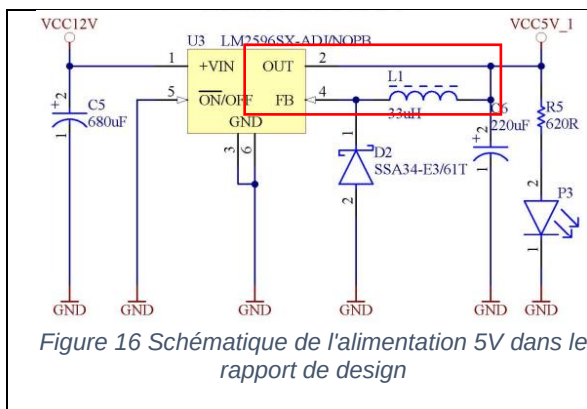
Bucher Mathieu

7. Modifications de la schématique du design

7.1. Alimentation 5V

Après la présentation du rapport de design, M. Moreno et M. Castoldi ont relevés des erreurs dans la schématique que j'ai fournis. Pour commencer, dans le bloc d'alimentation 5V, j'ai inversé la pin feedback et la pin out du LM2596. L'inversion de ces 2 pins supprime la régulation car L1 se trouve sur FB est n'est donc plus alimenté donc elle ne drive aucun courant.

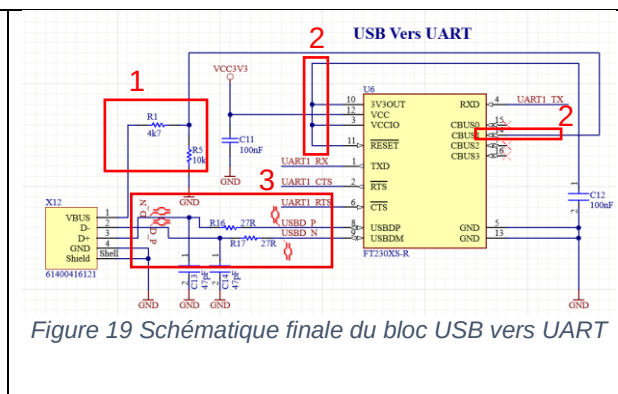
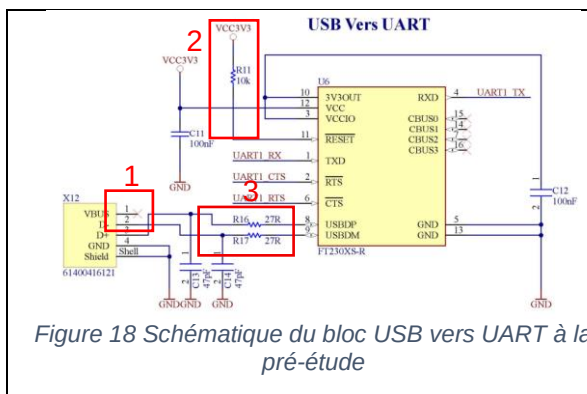
La broche FB sert de sensor pour la régulation de la tension de sortie. Dans le cas suivant FB se retrouve avec une entrée instable vu l'erreur en entrée qui varie entre 0V et Vin. FB n'est pas prévue pour supporter les surtensions et autres phénomènes causé par cette inversion.



J'ai aussi retiré la led de vie de l'alimentation car en soit elle n'est pas forcément nécessaire. Ces modifications sont faites sur les deux alimentations 5V.

7.2. Modifications du bloc USB vers UART

Il y a eu deux modifications dans le bloc :



1. J'ai ajouté un diviseur résistifs entre le VBUS du connecteur USB et la pin CBUS1 du FT230X qui permet de détecter le branchement de l'USB.
2. J'ai déconnecté la ligne reset de la résistance de pull-up car les lignes VCCIO et 3V3OUT sont à 3.3V, elles alimentent donc mon entrée reset directement.
3. J'ai rajouté une paire différentielle sur les lignes USBDP et USBDM, car les potentielles sont inversé sur les deux lignes, pour éviter les perturbation et parasites qui peuvent s'induire. Cela peut créer des erreurs de décodage.

7.3. Mesures du courant dans les moteurs

Comme l'ont précisé M. Moreno et M. Castoldi, une mesure du courant maximale tirés par un servo moteur était manquante. J'en ai donc profité pour la refaire pour confirmer mon choix d'alimentation.

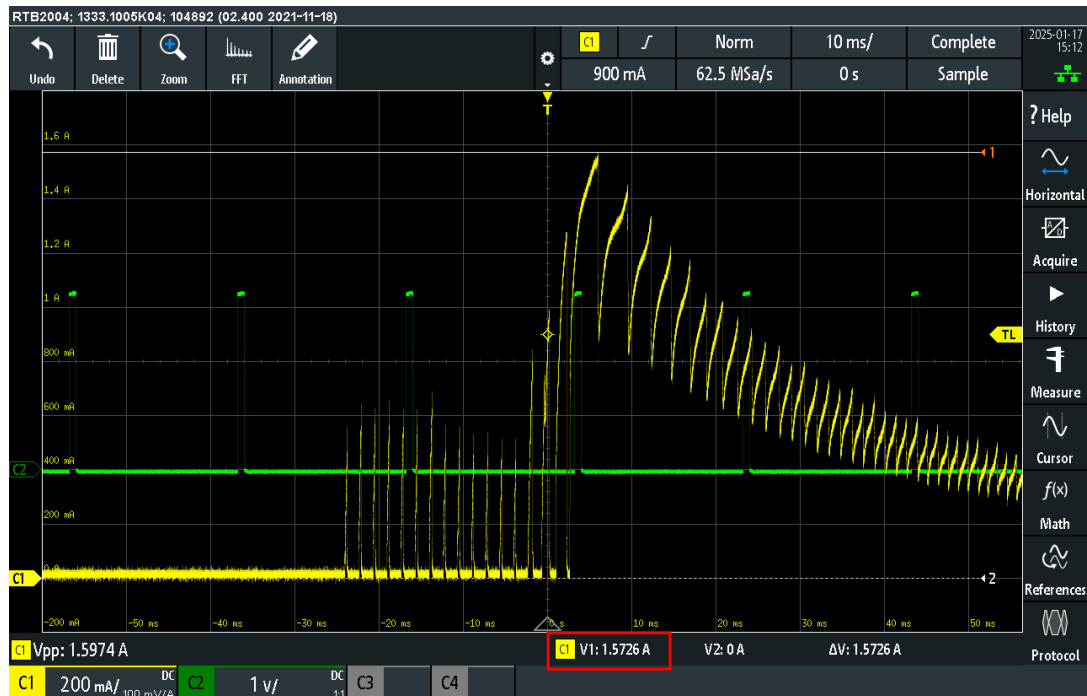


Figure 20 Mesure du courant maximale dans un servo moteur

Lorsque mon servo moteur se met en rotation, il tire un courant avec un pic maximale de 1.57 A. Etant donné que j'ai deux servos moteurs, j'aurai pu choisir un gros régulateur 5V avec 4 ou 5 ampères, mais il n'y avait que très peu d'options et ce choix serait beaucoup trop coûteux car ce type d'alimentation coûte aux alentours de 40 à 50 CHF.

7.4. Modification dans le bloc du contrôleur du plateau résistif

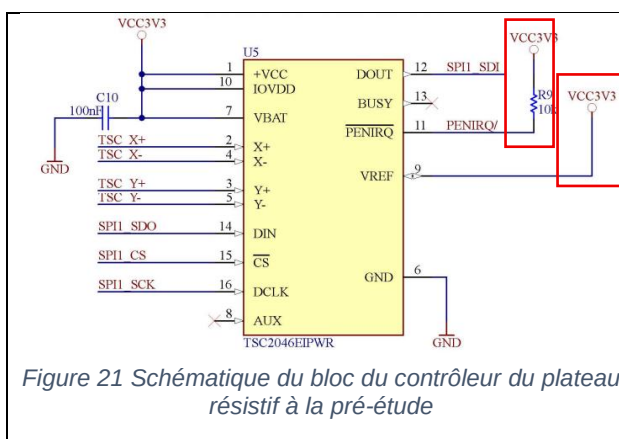


Figure 21 Schématique du bloc du contrôleur du plateau résistif à la pré-étude

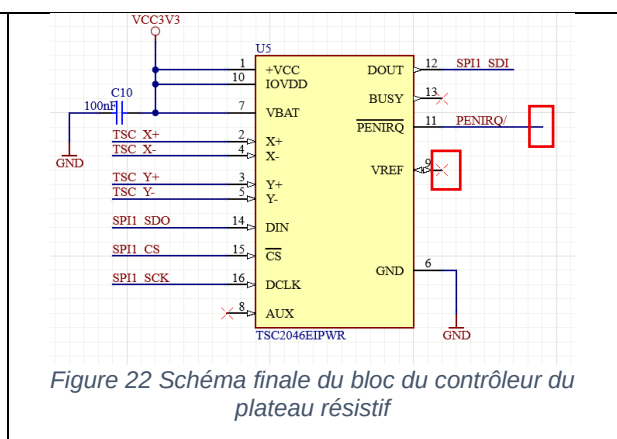


Figure 22 Schéma finale du bloc du contrôleur du plateau résistif

Dans cette partie il n'était pas nécessaire de connecter VREF au 3.3V car cette pin est connecté à +VCC de manière interne. C'est pareil pour la pin PENIRQ_, elle est déjà pull-up avec une résistance de 50kΩ en interne au TSC2046EPWR.

8. Hardware

8.1. PCB

En ce qui concerne le PCB, j'ai la chance de n'avoir aucune contrainte au niveau des dimensions à respecter et du placement de certains composants. Je me suis donc fixé une limite de taille pour le PCB de 130x100 mm pour la grandeur de mon PCB.

8.1.1. Placement

En ce qui concerne le placement, les principales « restrictions » seraient le placement du connecteur d'alimentation qui doit être accessible et au bord de la carte si l'on venait à créer un boîtier pour le projet. Le connecteur pour le plateau doit se situer le plus au bord de la carte aussi. J'ai ajouté des picots de GND aux quatre coins de la carte pour brancher la masse de mes sondes pendant d'éventuelle mesure. J'ai aussi rajouté 4 trous de fixation aux quatre coins.

J'ai séparé mon PCB en plusieurs blocs comme sur le schéma. Un bloc d'alimentation 12V, deux blocs d'alimentation 5V, le bloc 3.3V, le bloc avec le contrôleur du plateau résistif, le bloc avec le microcontrôleur et le bloc avec l'USB/UART.

8.1.2. Routage

Pour le routage il a fallu faire attention à la largeur de piste pour les alimentations surtout pour les deux lignes du 5V pour les moteurs. Si les pistes ne sont pas assez larges, et que le courant tiré est important elles risquent de sauter. J'ai utilisé le graphique disponible sur eurocircuit pour dimensionner mes pistes. J'ai augmenté les valeurs pour avoir un peu plus de marges même si de manière générale le circuit ne risque rien.

Liens Euro Circuit : <https://www.eurocircuits.com/technical-guidelines/pcb-design-guidelines/track-width-graphic/>

Pour le 12V, j'ai retrouvé une largeur de piste de 1.5 mm pour un courant de +2 A :

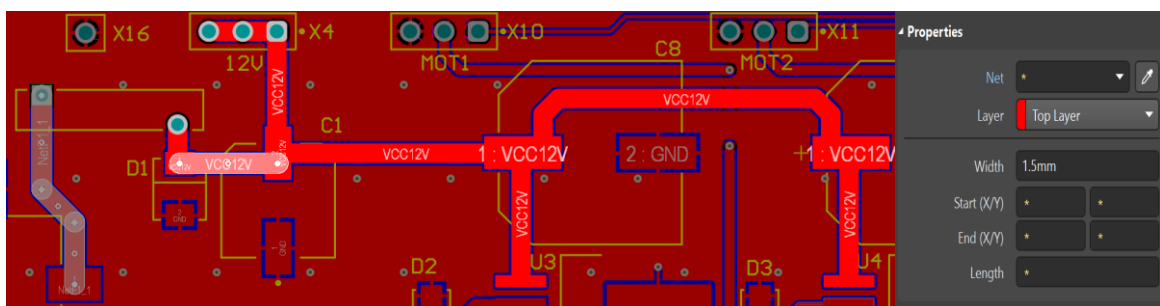


Figure 23 Largeur des pistes 12V

Pour les pistes 5V je sais que les servos moteurs peuvent tirer 1.57 ampères, alors en prenant un peu de marges, sur le graphique je retrouve une largeur de piste de 1.5mm :

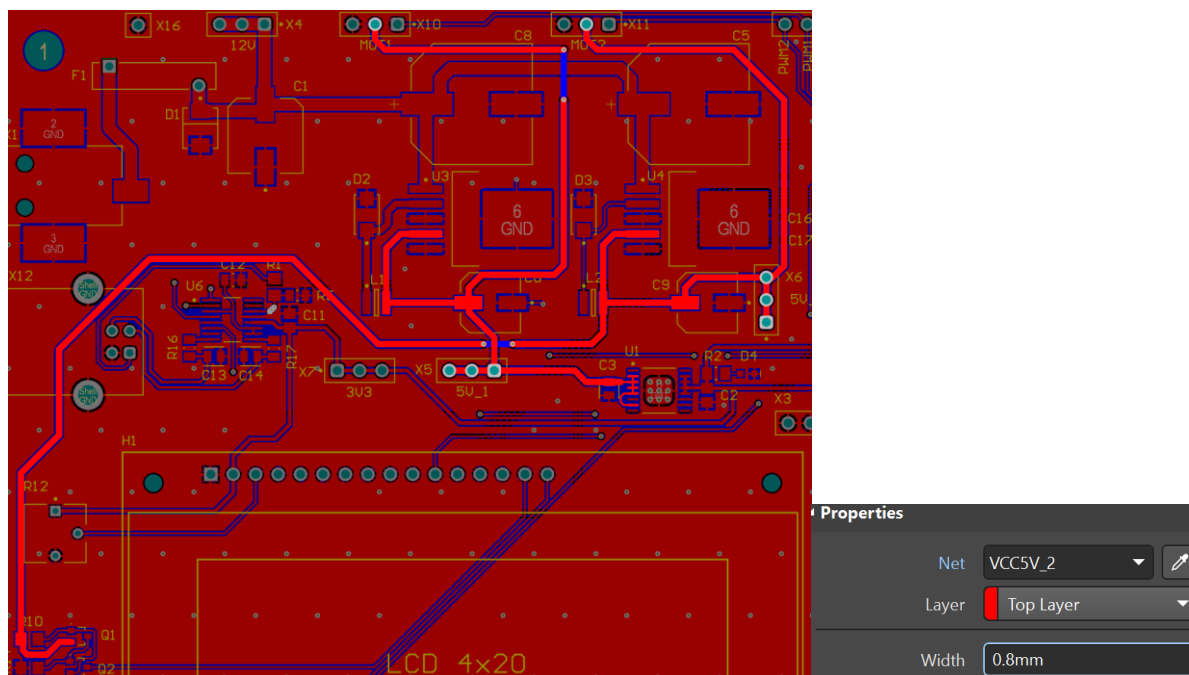


Figure 24 Largeur des pistes 5V_1 et 5V_2

Pour la ligne 3.3V, je trouve x mm pour x A :

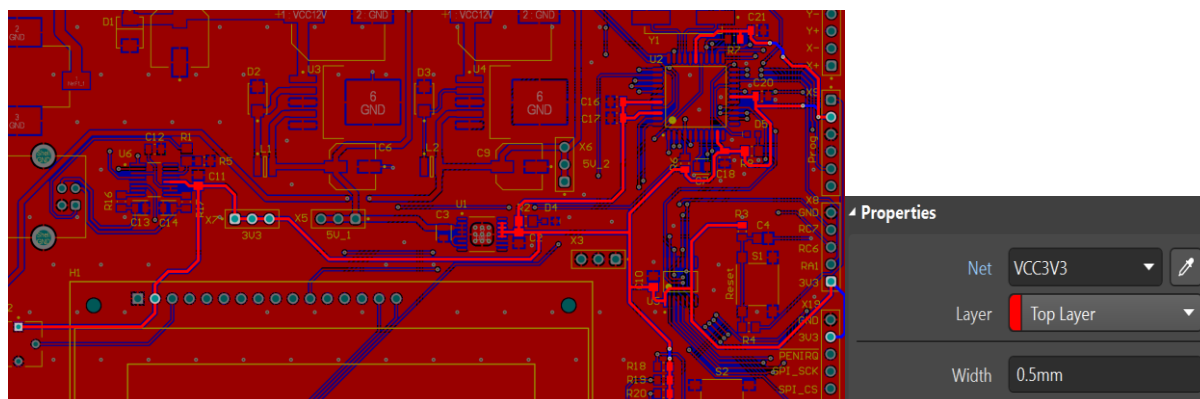


Figure 25 Largeur des pistes 3.3V

Pour le reste des pistes de signaux, j'ai choisi une largeur de piste standard de 0.3mm car le courant y circulant est relativement faible et ne causerait aucun risque.

Pour les deux lignes USB_D_P et USB_D_N, j'ai fait une paire différentielle pour garantir une communication précise entre l'ordinateur et mon FT230X.

8.1.3. 2D

Voici le rendu en 2D de mon PCB :

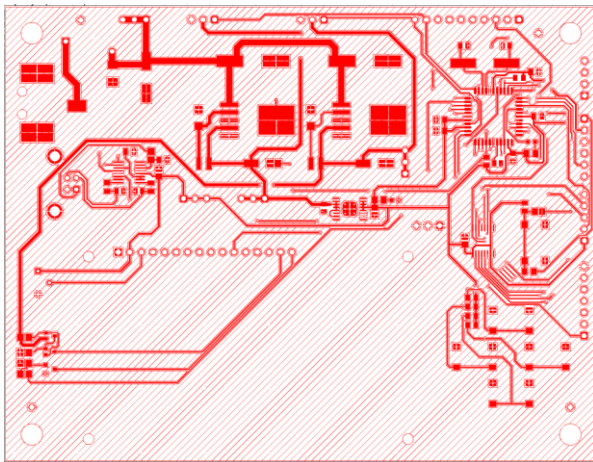


Figure 26 Vue 2D Top layer

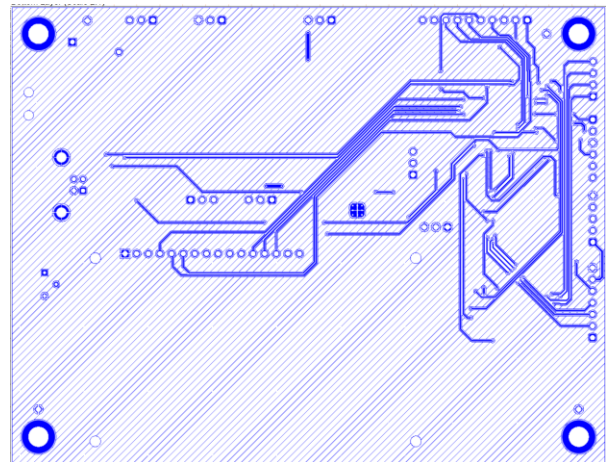


Figure 27 Vue 2D Bottom layer

8.1.4. 3D

Voici le rendu en 3D de mon PCB :

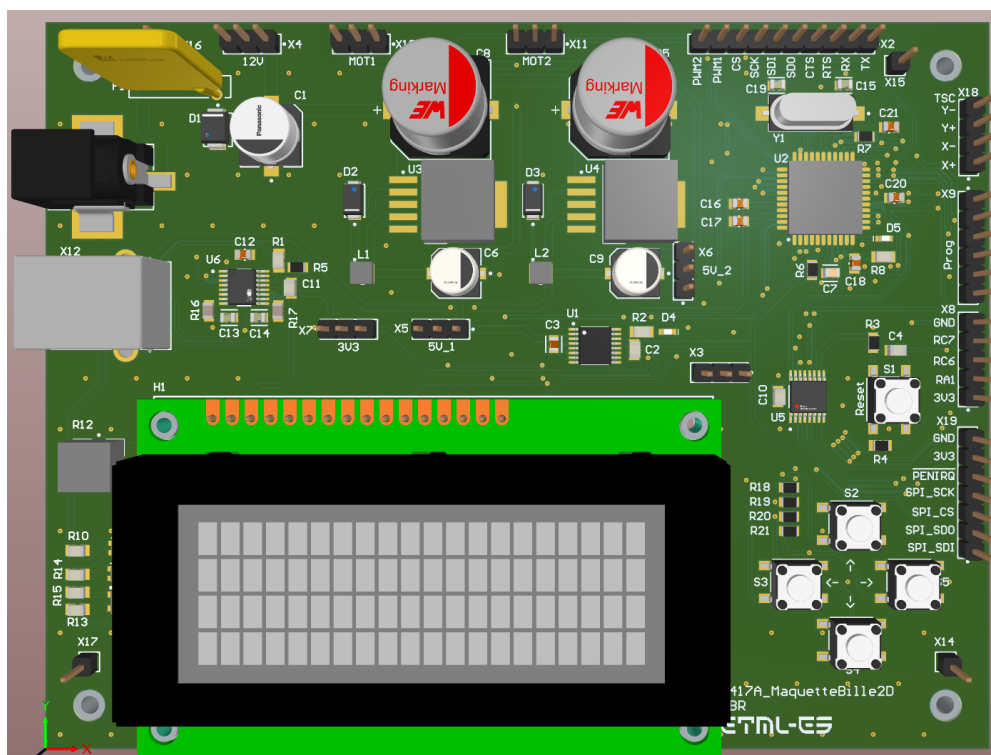


Figure 28 Rendu 3D du PCB

8.1.5. PCB monté

Voici le résultat de mon PCB complètement monté :



Figure 29 Photo du PCB monté

9. Correction hardware

9.1. Alimentation 5V

Pour l'alimentation, il y a une légère correction. Dans la schématique et la BOM du PCB, j'ai pris la version ajustable du LM2596. Il faut bien faire attention à utiliser la version 5V fixe.

LM2596SX-ADJ → LM2596S-5.0

9.1.1. Inductances

Dans la partie qui permet de générer une tension de 5V pour alimenter certains IC et les deux servomoteurs. J'ai choisi une inductance SRN80400TA-330MCT 33µH 2.3A qui supportait au maximum 500 mA. Les servos moteurs peuvent tirer jusqu'à 1,57A.

J'ai donc choisi une nouvelle inductance x.x pour remplacer mon premier choix. Cette inductance supporte un courant maximum de 2.1 A ce qui me donne un peu de marge.

Ensuite, j'ai effectué la correction sur mon PCB. Etant donné que l'inductance de remplacement est plus grande que le premier choix, son footprint initial ne convient pas, j'ai donc placé les inductances à la verticale.

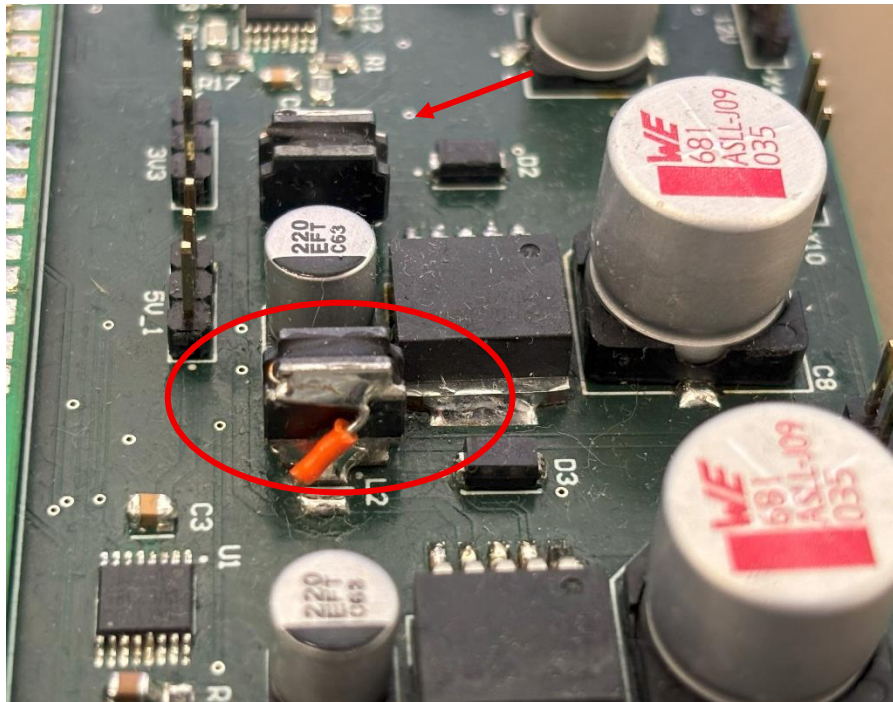


Figure 30 Correction sur le PCB avec la nouvelle inductance

9.2. Connection plateau résistif

Les pins de connexions du plateau résistifs sont intervertis. En effet, les pins X+, X-, Y+ et Y- ne se suivent pas.

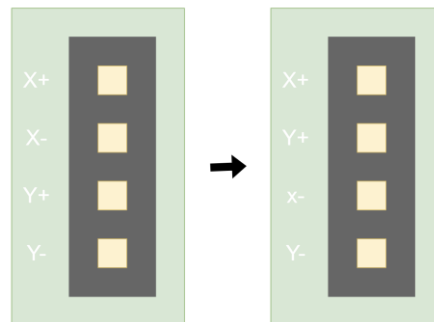


Figure 31 Correction de la connectique du plateau

Il a donc fallu modifier le câblage pour obtenir les bonnes valeurs venant du plateau résistif.

Le plateau fonctionne de la manière suivante :

Quand il n'y a aucun contact sur le plateau, on peut mesurer une valeur fixe entre les deux pôles X ou Y. Quand il y a une pression sur le plateau, la résistance entre deux pôles varie permettant aux TSC2046 de mesurer la tension sur cette résistance via l'ADC.

Quand il y a une pression dans un coin, la valeur de résistances entre deux pôles est soit maximale soit minimale.

10. Mise en service et mesures

J'ai monté petit à petit la carte et j'ai fait des mesures pour tester les alimentations, la communication des valeurs sur le plateau vers le microcontrôleur, la communication en UART vers l'USB, le PWM vers les servomoteurs.

10.1. Tension d'alimentation

10.1.1. Alimentation 3.3V

J'ai commencé par vérifier la tension de sortie sur mon régulateur 3.3V. Pour mesurer la tension je place ma sonde d'oscilloscope sur les pins de mesure X7.

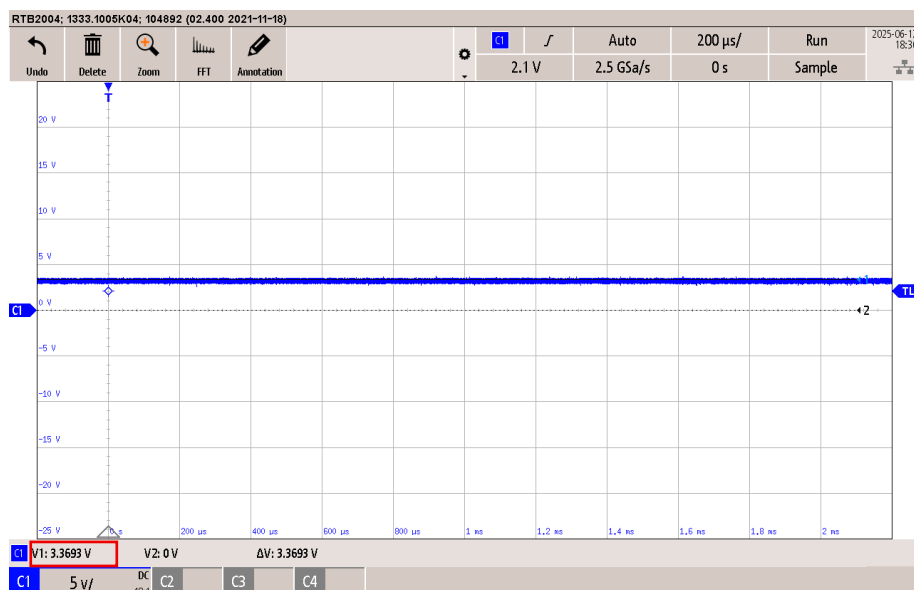


Figure 32 Mesure de la tension de sortie du régulateur 3.3V

La mesure est bonne, je mesure une tension de 3.693V sur la sortie du régulateur ce qui est largement tolérable.

10.1.2. Alimentation 5V

Pour les alimentations 5V c'est le même procédé, je viens placer ma sonde d'oscilloscope sur les points de mesure X5 pour le 5V_1 et X6 pour le 5V_2.

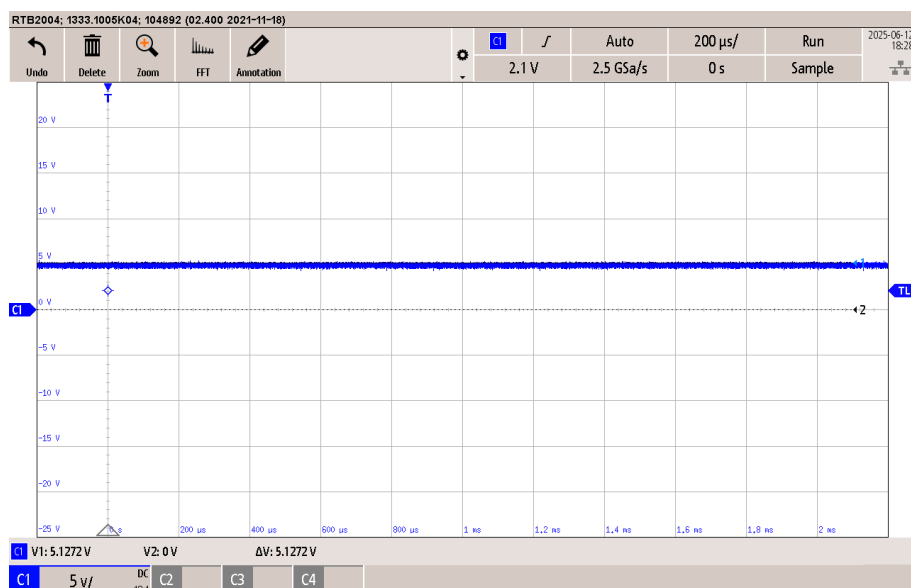


Figure 33 Mesure de la tension de sortie du régulateur 5V_1

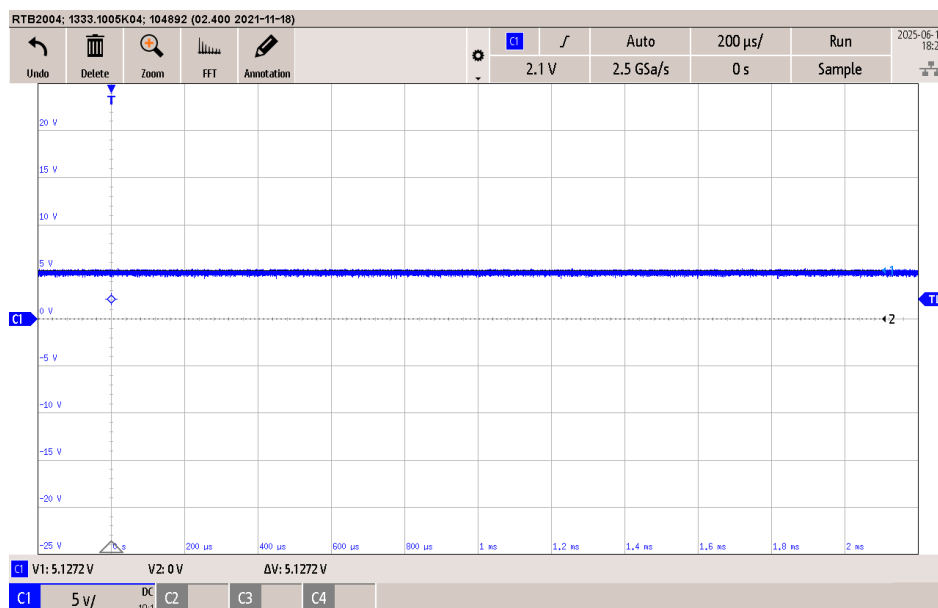


Figure 34 Mesure de la tension de sortie pour le 5V_2

10.1.3. Alimentation 12V

Toujours la même chose pour l'alimentation 12V. Une mesure sur les picots X4 pour obtenir :

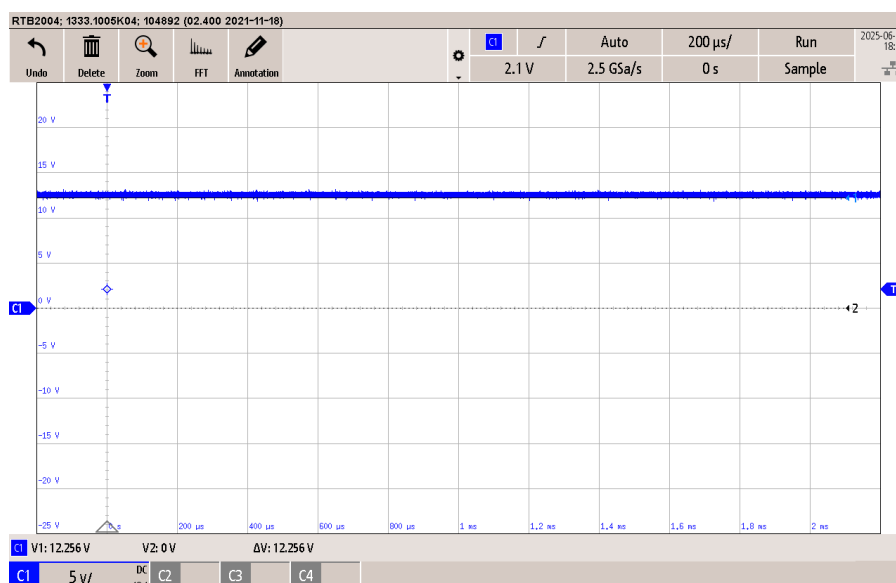


Figure 35 Mesure de la tension 12V en entrée de la carte

10.2. Mesure communication SPI entre le contrôleur et le microcontrôleur

J'ai effectué une mesure de mes signaux de communication SPI et j'ai comparé les valeurs obtenues sur les trames avec les valeurs brutes que je récupère sur le microcontrôleur.

Voici comment sont câblé le plateau et la carte :

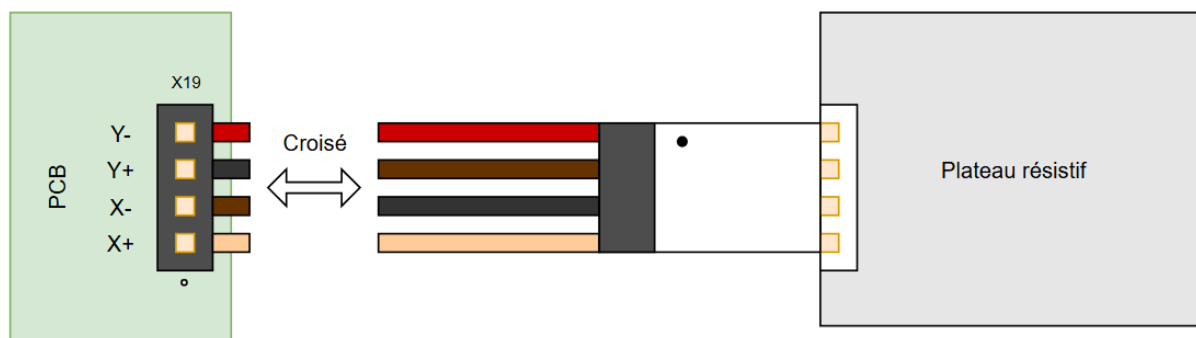


Figure 36 Schéma de câblage du plateau vers sur le PCB

J'ai ensuite créé une fonction de lecture `TSC2046_ReadRaw` qui permet de communiquer entre le TSC2046 et le microcontrôleur.

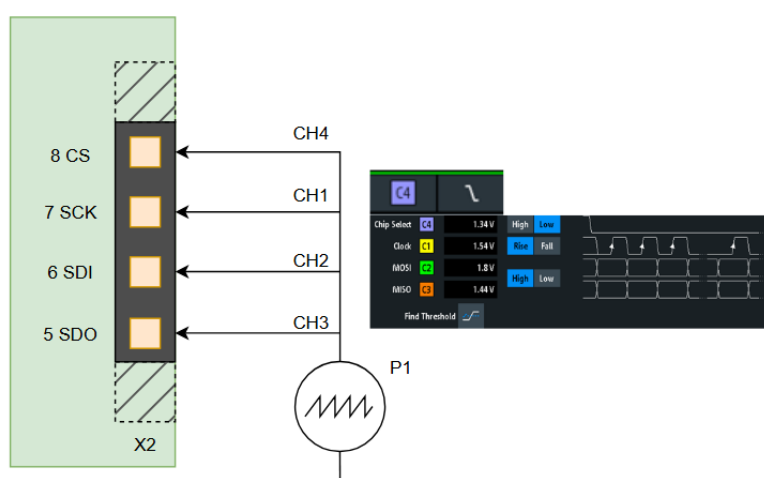


Figure 37 Schéma de mesure du SPI

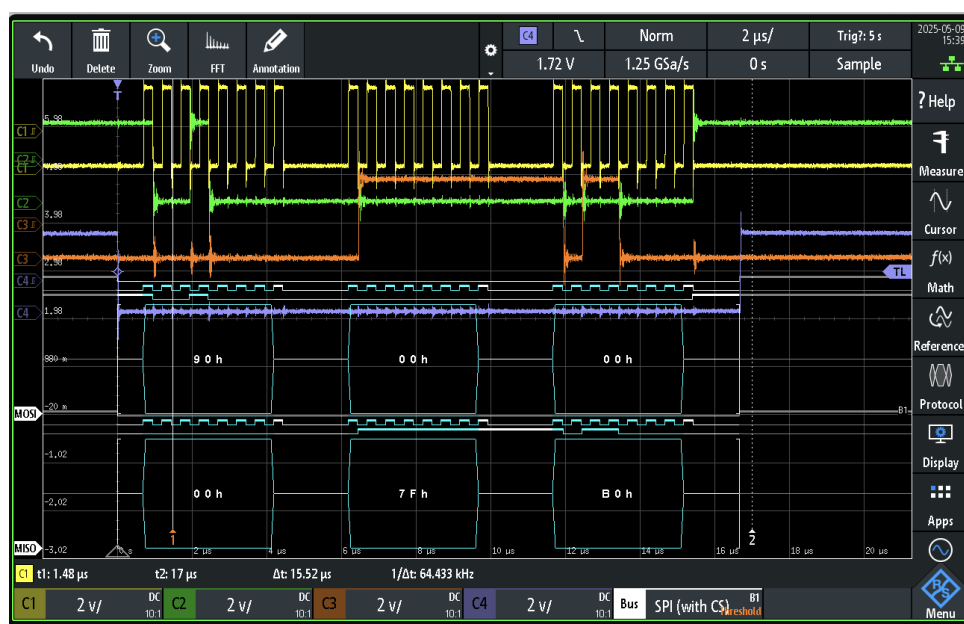


Figure 38 Mesure du SPI

Sur MPLABX, en faisant un « watch » de la valeur stockée dans la variable appData.rawY par exemple

10.3. Mesures des signaux PWM

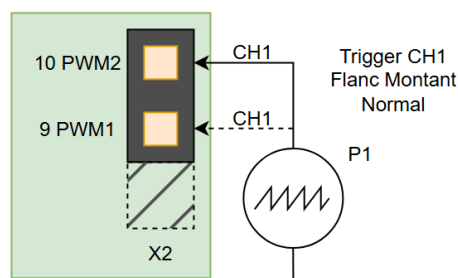


Figure 39 Schéma de mesure PWM1 et PWM2

J'ai mesuré les signaux PWM1 et PWM2 pour voir s'ils répondent bien à mes attentes. J'ai donc mesuré mes signaux à -90° (600 ms), 0° (1500ms) et $+90^\circ$ (2400ms).

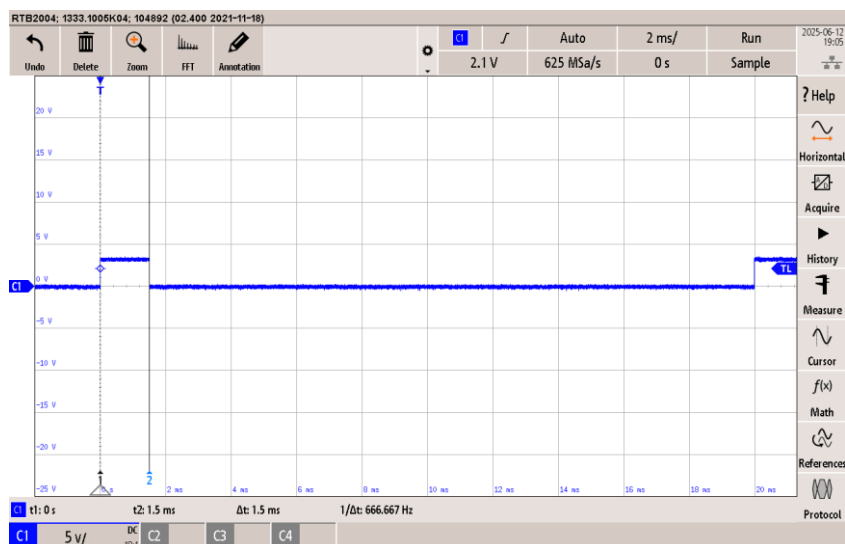


Figure 40 Mesure de la largeur d'impulsion à 0°

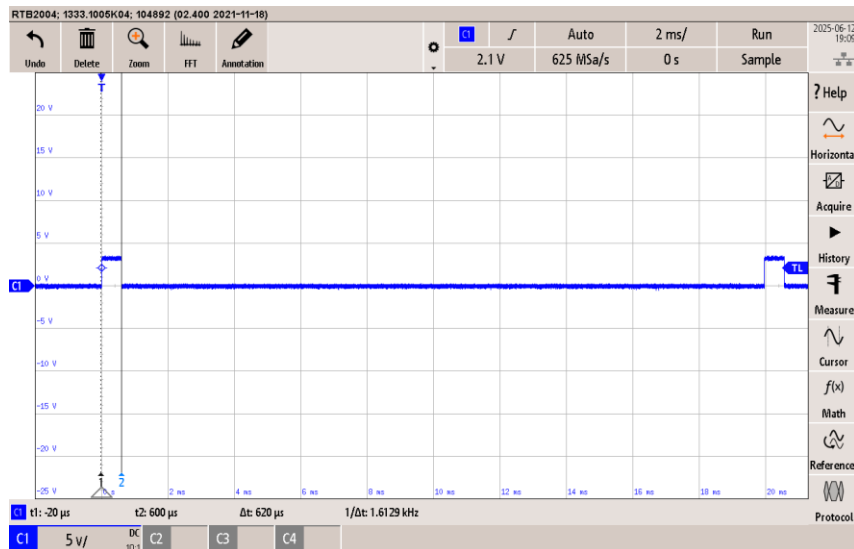


Figure 41 Mesure de la largeur d'impulsion à -90°

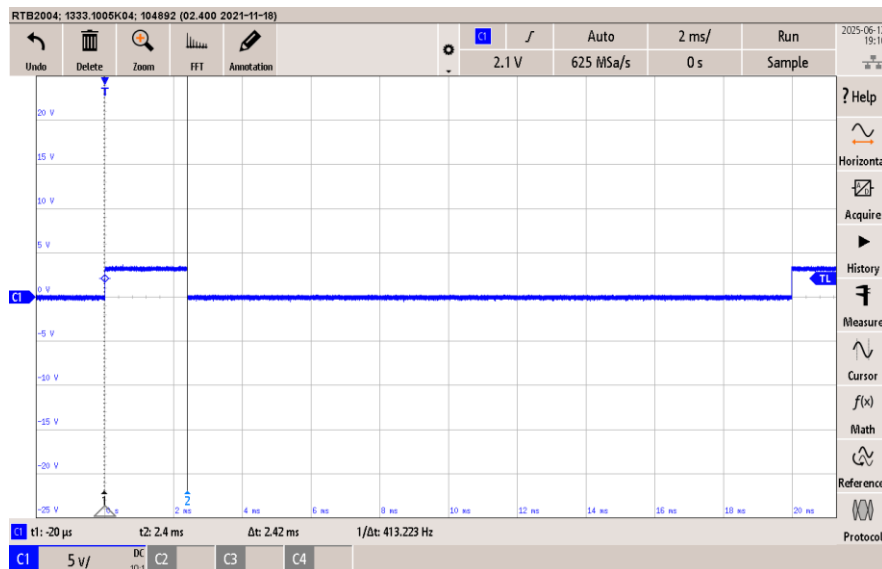


Figure 42 Mesure de la largeur d'impulsion à $+90^\circ$

La largeur d'impulsion de mes signaux respecte bien les valeurs attendues de 600 μ s pour -90° , 1500 μ s pour 0° et 2400 μ s pour $+90^\circ$.

10.4. Mesure de la communication UART

J'ai fait deux tests, le premier en envoyant une trame depuis le microcontrôleur et le second en envoyant la trame depuis l'ordinateur via Putty. Voici la configuration utilisée pour Putty :

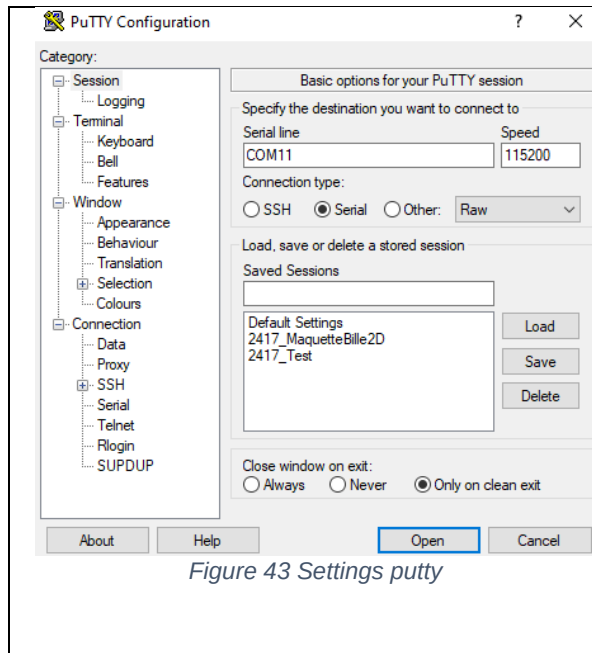


Figure 43 Settings putty

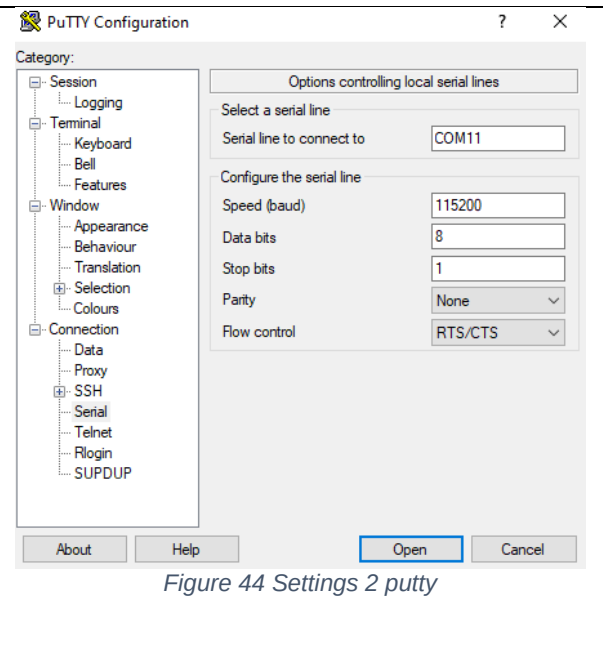


Figure 44 Settings 2 putty

J'ai d'abord testé l'envoi de la trame pour la position en temps réel de la bille :

```
!X=0633Y=0713#
!X=0657Y=0739#
```

Figure 45 Position de la bille sur le plateau résistif

10.5. Courant

10.5.1. Consommation totale

J'ai ensuite mesuré le courant passant dans le circuit avec une sonde de courant. J'ai utilisé un câble d'alimentation jack avec le VCC et le GND séparés sur une alimentation 12V. et j'ai connectés la sonde de courant sur le VCC du câble.

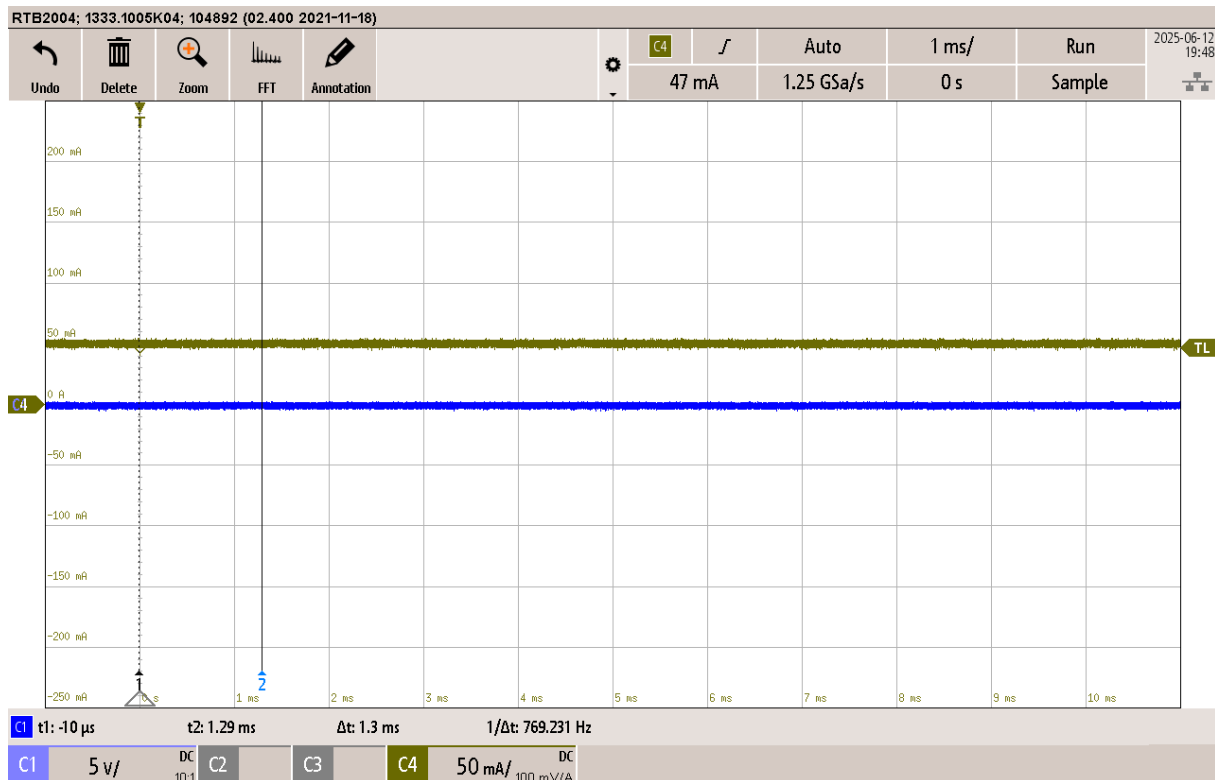


Figure 46 Courant sans fonctionnement des moteurs

Ma carte consomme 50 mA lorsqu'aucun des moteurs n'est en fonction. Lorsque les moteurs se mettent en mouvement le courant augmente jusqu'à atteindre des valeurs avoisinant les 2,5 A car il ne tourne pas à fond grâce aux facteurs de correction.

11. Firmware

11.1. Fonctions

11.1.1. Lecture plateau

La lecture de la position sur le plateau se fait avec le TSC2046E et communiquant en SPI vers le microcontrôleur. J'ai récupéré la librairie utilisée pour les TP de MINF « Mc32GestSPI.c et .h ». Cette librairie contient le nécessaire pour lancer la communication. Il a fallu faire attention à la polarité des signaux.

Image polarité

Je fais une lecture de la valeur en dure qui doit varier entre 0 et 4096. Je récupère cette valeur et je le stock dans une structure. Ensuite je convertis cette valeur en pixel.

Les fonctions `TSC2046_ReadRawX()` et `TSC2046_ReadRawY()` permettent d'enregistrer la valeur récupérer depuis le contrôleur dans une variable 16 bits endécalant de 4 bits car les 4 derniers bits sont ignoré puisque le contrôleur à une résolution de 12 bits

11.1.2. Réglage P

La fonction `PRegulation()`

$$pulse = SERVO_PWM_NEUTRAL_TICK \cdot kp \cdot erreur \cdot direction$$

Je calcul l'erreur en soustrayant la valeur mesurée brute et la valeur de référence zero qui correspond a l'endroit où la bille doit se placer (le centre). Cette erreur représente le décalage de la bille par rapport au centre.

$$erreur = valeurRaw - ZERO$$

Ensuite, je lui applique ensuite le gain k_p pour transformer cette erreur en mouvement du servomoteur (modifier l'angle). J'ajoute finalement ce résultat à l'impulsion pour avoir le plateau à plat. Si cette valeur ne change pas le servo moteur laisse le plateau à plat. Si l'écart se creuse, alors l'impulsion s'allonge ou se raccourcit de manière proportionnelle pour recentrer la bille. Je donne aussi la direction pour obtenir le sens de l'axe (un des axes est inversé)

$$pulse = SERVO_PWM_NEUTRAL_TICK \cdot k_p \cdot erreur \cdot direction$$

J'ai aussi une vérification des valeurs pour éviter de dépasser la largeur d'impulsion maximale et minimale.

11.1.3. Réglage I

L'intégrale permet d'annuler l'erreur que laisse la correction proportionnelle.

La fonction `IRegulation()` récupère la valeur brute et recalcule l'erreur en fonction du zéro (balle au centre)

$$erreur = direction \cdot (valeurRaw - ZERO)$$

Le facteur direction règle le sens de rotation du moteur pour l'axe inversé. Il faut ensuite intégrer l'erreur.

$$Integ += k_i \cdot erreur \cdot Ts$$

A chaque fois, l'intégrale calcule et réajoute à la dernière valeur. Quand l'erreur grandit, l'intégrale aussi et allonge l'impulsion jusqu'à ce que la balle revienne au centre. Il y a de nouveau une vérification pour ne pas dépasser les valeurs limite du PWM.

11.1.4. Réglage D

La dérivation permet de freiner la balle si elle se déplace trop rapidement.

La fonction `DRegulation()` va atténuer le dépassement provoqué par la correction proportionnelle. Toujours le même principe pour recalculer l'erreur.

La pente est obtenue par la différence entre l'erreur actuelle et la précédente divisée par la période d'échantillonnage. Il faut alors multiplier cette pente par k_d pour obtenir la dérivation.

$$Dérivation = k_d \cdot \frac{erreur - erreurPrécédente}{Ts}$$

11.1.5. Règlage PID

La fonction `PIDRegulation()` réunit les trois corrections pour générer la largeur d'impulsion finale pour le servo moteur de l'axe en question.

$$pulsePIDReg = P + I + D$$

Ce qui nous donne une valeur en tick avec une vérification de dépassement de valeur maximale et minimale.

11.1.6. Génération PWM

Avec la valeur Raw qui est ensuite régulée, j'obtiens une valeur allant de 450 à 1800 qui permet de générer le PWM pour contrôler le servomoteur.

Pour générer le PWM, j'utilise la fonction de la PLIB « `PLIB_OC_PulseWidth16BitSet(OC_ID_1ou2, pwmXouY)` » en donnant lequel des OC je souhaite utiliser et la valeur régulée pour l'axe X ou Y.

11.1.7. Envoie UART

La fonction `sendPIDXY()` e

Met en forme la trame en divisant par 1000 les facteurs et en les insérant dans un buffer avec `sprintf()` sous la forme `!kp=xxxxki=xxxxkd=xxxxX=xxxxY=xxxx#`. Il y a une mise en mémoire de la longueur de la trame avec `len`. Il y a ensuite une vérification du CTS pour savoir si le FT230 est prêt. Puis une vérification de la FIFO pour savoir s'il reste de la place. Enfin, l'écriture de la trame se fait dans le buffer d'envoi de l'UART.

La fonction `sendPosition()` effectue l'envoi de la trame de la même manière que la fonction `sendPIDXY()` mais cette fois en envoyant simplement une trame mais cette fois ne contenant que la position actuelle de la bille : `!X=xxxxY=xxxx#`.

11.1.8. Réception UART

La fonction `ReceiveGain()` permet de mettre à jour les facteurs envoyés par l'utilisateur depuis le PC.

En faisant une lecture de la FIFO de réception, la fonction va récupérer chaque caractère un par un. Elle s'arrête quand la FIFO est vide.

Il y a un état d'attente qui attend de réceptionner le caractère « ! ». Une fois ce caractère récupéré on rentre dans la réception et on stocke chaque octet dans jusqu'à ce qu'on obtienne le « # ».

Ensuite, avec un `sscanf()`, une recherche des valeurs entières entre les k= s'effectue : `"kp=%4u ki=%4u kd=%4u"`. Si le format de la trame n'est pas bon alors elle est ignorée.

11.1.9. Gestion des boutons

La fonction `lectureBouton()` permet de lire l'état des boutons up, down, left, right.

La fonction `boutonsDetectorFront()` permet de détecter la pression d'un bouton avec une mise en mémoire de l'état du bouton et son état précédant.

La fonction `menuGestionBouton()` s'occupe de tout le traitement entre l'interaction avec les boutons et le menu. Elle lit l'état des touches à chaque appel. Elle est constituée par trois états :

MENU_OFF : une pression sur bouton droite dans cet état bascule vers MENU_BROWSE.

MENU_BROWSE : Permet de défiler le curseur entre kp, ki et kd avec bouton up et down. Une pression sur droite entre dans l'édition des digits(MENU_EDIT). La touche gauche quitte l'édition (retour à MENU_BROWSE) ou ferme le menu(retour à MENU_OFF).

MENU_EDIT : Dans cet état, on modifie le gain chiffre par chiffre. Haut augmente le digit et bas décrémente le digit. Droite et gauche permettent de sélectionner le digit. Si le curseur est tout à droite retour au premier digit. Si tout à gauche ferme l'édition et retourne dans MENU_BROWSE. Les gains sont affichés sous cette forme « 0.000 ».

11.1.10. Affichage

La fonction `menuAffichage()` permet de rafraîchir l'écran LCD.

La fonction `lcdShowPos()` affiche les valeurs X et Y en pixel de la bille sur l'écran LCD.

12. Software

La partie software est manquante par manque de temps. Il y a déjà un début d'application mais pas du complet. Je préfère présenter quelque chose de complet et fonctionnel pour cette partie. Je vais essayer de rendre le software fonctionnel pour la présentation.

13. Amélioration

13.1.1. Chose fonctionnelle à améliorer

J'ai pu tester la réception de la trame depuis Putty vers ma carte, elle était fonctionnelle. L'envoi de la trame avec la position est fonctionnelle et l'envoi de la trame avec la dernière position et les facteurs de réglages étaient fonctionnelles à un moment. Le problème est que je n'arrive pas à les implémenter les 3 ensembles. Il faut que je fasse fonctionner ça pour la présentation.

Je devrais ajouter un écran d'accueil quand je mets la carte sous tension.

13.1.2. Choses manquantes à ajouter

Il faut tout d'abord ajouter une fonction qui permette de détecter quand la balle est posée sur le plateau ou non. Je ne l'ai pas faite car je m'y suis pris trop tard. La pin PENIRQ du TSC2046 passe à l'état bas quand la balle est posée. Le problème est que pendant la lecture des valeurs X et Y, la pin PENIRQ change d'état en fonction de la lecture de X+, X-, Y+ et Y-.

Je dois compléter l'application C# et la faire fonctionner avec mon projet car je n'ai pas eu le temps de travailler dessus.

Ajouter de la sérigraphie pour le sens des connecteurs des moteurs.

14. Aspect environnemental du projet

Ce projet reste un dispositif de laboratoire de petite puissance. Sa consommation quand aucun moteur ne tourne ne dépasse pas les 60mA. Lorsque les moteurs tournent sa consommation de courant peut avoisiner au maximum les 3 ampères pendant un court

instant et retourne rapidement en dessous d'un ampère. Les composants choisis respectent les normes RoHS II. Les deux régulateurs Buck en un bon rendement et dissipe largement la chaleur.

15. Aspect social

Ce projet a pour but de servir d'outils pédagogique pour l'apprentissage et la compréhension du cours de réglage. Il est fait pour être accessible à toute types d'utilisateurs et pas seulement aux étudiants en électronique. Grâce à l'interface physique ou même l'interface de l'application, le projet poussent à apprendre par la pratique. De plus le projet peut être utilisé dans le cadre d'un TP par exemple pour apprendre à réaliser de la régulation et utiliser différents bus de communication. Le projet est assez sécuritaire car il est assez difficile d'alimenter la carte en inverse.

16. Aspect économique

Le projet n'est pas voué à être commercialisé, le développement est de plus trop coûteux. C'est un projet à produire en peu d'exemplaire uniquement à but pédagogique.

17. Conclusion

Pour commencer, j'ai corrigé les erreurs faites sur le schéma lors de la phase de pré-étude et de design. J'ai eu quelques modifications qui provenaient d'erreurs d'inattentions surtout. Je les ai corrigées et j'ai pu me lancer sur la conception du PCB.

Lors de la conception du PCB, j'ai eu la chance de n'avoir aucune contrainte mécanique à respecter ce qui m'a facilité la tâche.

Lors de la phase de montage j'ai remarqué que les inductances que j'ai choisies ne supportait pas le courant que pouvaient tirer mes servos moteurs. J'ai aussi pu constater qu'en testant mon alimentation 5V, j'avais commandé la version ajustable du LM2596. J'ai alors refait une commande de composants en choisissant une inductance supportant plus de courant et en prenant la version 5V fixe du LM2596. J'ai pu ensuite monter mon PCB complètement sans aucun problème.

Pour le firmware, j'ai commencé par tester la lecture des valeurs sur le plateau résistif et pour ce faire j'ai commencé en communiquant en SPI vers le contrôleur. J'ai perdu énormément de temps sur cette partie car la connectique de mon écran était fautive. J'ai donc interverti le câblage pour avoir les bons signaux sur les entrées du contrôleur. Ensuite, j'ai généré les signaux PWM en fonctions de la position de la bille. Cette partie était assez rapide à faire. Une chose qui m'a pris énormément de temps, était la création des fonctions de réglages. Cela m'a pris plus de temps que prévu mais j'ai pu m'en sortir. La communication UART m'a pris un peu de temps car je n'ai pas pu faire les tests tout de suite car mon ordinateur n'avait pas les drivers FTDI installés. La communication UART est fonctionnel mais pas en mettant ensemble les trois types de trames que je souhaite envoyer et recevoir. La réception d'une trame venant de l'ordinateur vers ma carte est fonctionnelle j'ai réussi à modifier les facteurs. L'envoi d'une trame avec la position en temps réel de la bille fonctionne aussi. Et l'envoi d'une trame envoyant la dernière position de la bille et les facteurs de réglages fonctionne. Mon souci est que si je mets ensemble ces trois types de trames mon programme bloque. Je ne peux envoyer la trame depuis putty car j'envoie en continue la position et je sature l'UART. Je vais essayer de régler ce problème pour la présentation.

Pour l'application en C#, je n'ai pas eu le temps de la compléter et de la rendre fonctionnelle.

Dans l'ensemble, le projet se passait bien. Mais en se rapprochant du délai de rendu, j'ai commencé à me disperser en essayant de compléter l'entier du cahier des charges plutôt qu'en me concentrant sur les points les plus importants. J'ai mal géré mon temps et j'ai du rendre un travail bâclé sur la fin.

Lausanne le 13.06.2025,

Bucher Mathieu

18. Annexes

18.1. Datasheets

OPTICAL AND ELECTRICAL CHARACTERISTICS ($T_{amb} = 25\text{ }^{\circ}\text{C}$, unless otherwise specified) VLMY1300, VLMY1301, YELLOW							
PARAMETER	TEST CONDITION	PART NUMBER	SYMBOL	MIN.	TYP.	MAX.	UNIT
Luminous intensity	$I_F = 20\text{ mA}$	VLMY1300	I_V	28	-	180	mcd
		VLMY1301	I_V	71	-	180	mcd
Dominant wavelength	$I_F = 20\text{ mA}$		λ_d	584.5	-	597	nm
Peak wavelength	$I_F = 20\text{ mA}$		λ_p	-	588	-	nm
Angle of half intensity	$I_F = 20\text{ mA}$		ϕ	-	± 65	-	deg
Spectral line half width	$I_F = 20\text{ mA}$		$\Delta\lambda$	-	15	-	nm
Forward voltage	$I_F = 20\text{ mA}$		V_F	1.8	-	2.4	V
Junction capacitance	$V_R = 0\text{ V}$, $f = 1\text{ MHz}$		C_j	-	40	-	pF
Reverse current	$V_R = 5\text{ V}$		I_R	-	-	10	μA

Figure 47 Datasheet VLMY1300

Electrical Characteristics at $T_A=25^{\circ}\text{C}$

Part Number	Color	Forward Voltage V_F (Volts) @ $I_F = 20\text{ mA}$		Reverse Breakdown V_R (Volts) @ $I_R = 100\text{ }\mu\text{A}$	Capacitance C (pF), @ $V_F = 0\text{ V}$, $f = 1\text{ MHz}$	Thermal Resistance $R\theta_{J-P}$ ($^{\circ}\text{C/W}$)
		Typ.	Max.	Min.	Typ.	Typ.
HSMS-C110/150	HER	2.1	2.6	5	5	400
HSMS-C120						350
HSMS-C170/177/190/191/197						250
HSMD-C110/150	Orange	2.2	2.6	5	7	400
HSMD-C120						350
HSMD-C170/177/190/191/197						250
HSMY-C110/150	Yellow	2.1	2.6	5	6	400
HSMY-C170/177/190/191/197						250
HSMG-C110/150	Green	2.1	2.6	5	9	400
HSMG-C120						350
HSMG-C170/177/190/191/197/265						250
HSMH-C110/150	AlGaAs	1.8	2.6	5	18	460
HSMH-C120						400
HSMH-C170/190/191/265						300

Figure 48 Datasheet HSMD-C150

PIC32MX1XX/2XX 28/36/44-PIN FAMILY

TABLE 11-1: INPUT PIN SELECTION

Peripheral Pin	[pin name]R SFR	[pin name]R bits	[pin name]R Value to RPN Pin Selection
INT4	INT4R	INT4R<3:0>	0000 = RPA0 0001 = RPB3 0010 = RPB4 0011 = RPB15 0100 = RPB7 0101 = RPC7 ⁽²⁾ 0110 = RPC0 ⁽¹⁾ 0111 = RPC5 ⁽²⁾ 1000 = Reserved * * * 1111 = Reserved
T2CK	T2CKR	T2CKR<3:0>	
IC4	IC4R	IC4R<3:0>	
SS1	SS1R	SS1R<3:0>	
REFCLKI	REFCLKIR	REFCLKIR<3:0>	
INT3	INT3R	INT3R<3:0>	0000 = RPA1 0001 = RPB5 0010 = RPB1 0011 = RPB11 0100 = RPB8 0101 = RPA8 ⁽²⁾ 0110 = RPC8 ⁽²⁾ 0111 = RPA9 ⁽²⁾ 1000 = Reserved * * * 1111 = Reserved
T3CK	T3CKR	T3CKR<3:0>	
IC3	IC3R	IC3R<3:0>	
U1CTS	U1CTSR	U1CTSR<3:0>	
U2RX	U2RXR	U2RXR<3:0>	
SDI1	SDI1R	SDI1R<3:0>	
INT2	INT2R	INT2R<3:0>	0000 = RPA2 0001 = RPB6 0010 = RPA4 0011 = RPB13 0100 = RPB2 0101 = RPC6 ⁽²⁾ 0110 = RPC1 ⁽¹⁾ 0111 = RPC3 ⁽¹⁾ 1000 = Reserved * * * 1111 = Reserved
T4CK	T4CKR	T4CKR<3:0>	
IC1	IC1R	IC1R<3:0>	
IC5	IC5R	IC5R<3:0>	
U1RX	U1RXR	U1RXR<3:0>	
U2CTS	U2CTSR	U2CTSR<3:0>	
SDI2	SDI2R	SDI2R<3:0>	
OCFB	OCFBR	OCFBR<3:0>	
INT1	INT1R	INT1R<3:0>	0000 = RPA3 0001 = RPB14 0010 = RPB0 0011 = RPB10 0100 = RPB9 0101 = RPC9 ⁽¹⁾ 0110 = RPC2 ⁽²⁾ 0111 = RPC4 ⁽²⁾ 1000 = Reserved * * * 1111 = Reserved
T5CK	T5CKR	T5CKR<3:0>	
IC2	IC2R	IC2R<3:0>	
SS2	SS2R	SS2R<3:0>	
OCFA	OCFAR	OCFAR<3:0>	

Note 1: This pin is not available on 28-pin devices.

Note 2: This pin is only available on 44-pin devices.

PIC32MX1XX/2XX 28/36/44-PIN FAMILY

TABLE 11-2: OUTPUT PIN SELECTION

RPn Port Pin	RPnR SFR	RPnR bits	RPnR Value to Peripheral Selection
RPA0	RPA0R	RPA0R<3:0>	0000 = No Connect
RPB3	RPB3R	RPB3R<3:0>	0001 = U1TX
RPB4	RPB4R	RPB4R<3:0>	0010 = U2RTS
RPB15	RPB15R	RPB15R<3:0>	0011 = SS1
RPB7	RPB7R	RPB7R<3:0>	0100 = Reserved
RPC7	RPC7R	RPC7R<3:0>	0101 = OC1
RPC0	RPC0R	RPC0R<3:0>	0110 = Reserved
RPC5	RPC5R	RPC5R<3:0>	0111 = C2OUT
RPA1	RPA1R	RPA1R<3:0>	1000 = Reserved
RPB5	RPB5R	RPB5R<3:0>	*
RPB1	RPB1R	RPB1R<3:0>	*
RPB11	RPB11R	RPB11R<3:0>	1111 = Reserved
RPB8	RPB8R	RPB8R<3:0>	0000 = No Connect
RPA8	RPA8R	RPA8R<3:0>	0001 = Reserved
RPC8	RPC8R	RPC8R<3:0>	0010 = Reserved
RPA9	RPA9R	RPA9R<3:0>	0011 = SDO1
RPA2	RPA2R	RPA2R<3:0>	0100 = SDO2
RPB6	RPB6R	RPB6R<3:0>	0101 = OC2
RPA4	RPA4R	RPA4R<3:0>	0110 = Reserved
RPB13	RPB13R	RPB13R<3:0>	0111 = C3OUT
RPB2	RPB2R	RPB2R<3:0>	*
RPC6	RPC6R	RPC6R<3:0>	*
RPC1	RPC1R	RPC1R<3:0>	*
RPC3	RPC3R	RPC3R<3:0>	1111 = Reserved
RPA3	RPA3R	RPA3R<3:0>	0000 = No Connect
RPB14	RPB14R	RPB14R<3:0>	0001 = U1RTS
RPB0	RPB0R	RPB0R<3:0>	0010 = U2TX
RPB10	RPB10R	RPB10R<3:0>	0011 = Reserved
RPB9	RPB9R	RPB9R<3:0>	0100 = SS2
RPC9	RPC9R	RPC9R<3:0>	0101 = OC3
RPC2	RPC2R	RPC2R<3:0>	0110 = Reserved
RPC4	RPC4R	RPC4R<3:0>	0111 = C1OUT
			1000 = Reserved
			*
			*
			1111 = Reserved

18.2. Schéma

Voir feuilles Annexes.

18.3. Listing firmware

Voir feuilles annexes.

18.4. Prix PCB Eurocircuits

Liens du calculateur :

<https://be.eurocircuits.com/shop/assembly/configurator.aspx?sessionid=702e11a8-23f0-4062-9ea5-73e7c2b7478b>

Prices	Net	Gross*
Single PCB	€ 61.43	€ 74.33
Total boards	€ 61.43	€ 74.33
Total **	€ 61.43	€ 74.33

* The gross prices include 21 % VAT.

Figure 51 Prix suggérer pour un pcb de taille 130mm x 100mm